

Politechnika Warszawska

Wydział Matematyki
i Nauk Informacyjnych

Praca dyplomowa licencjacka

na kierunku Matematyka i Analiza Danych

Wpływ funkcji wag pewności na jakość rekomendacji gier w modelu implicit ALS

Michał Tałałaj

Numer albumu 333092

promotor

dr inż. Robert Małysz

WARSZAWA 2026

Streszczenie

Słowa kluczowe:

Abstract

Keywords:

Spis treści

Streszczenie **3**
Abstract **4**
1. Wstęp **11**
1.1. Motywacja i problem badawczy 11
1.2. Cel, zakres i pytania badawcze 11
1.3. Struktura pracy 11
2. Podstawy teoretyczne systemów rekomendacyjnych **12**
2.1. Zadanie rekomendacji Top-N 12
2.2. Klasyczna faktoryzacja macierzy dla danych jawnych 13
2.2.1. Model Funk SVD i funkcja celu 13
2.2.2. Ograniczenia tradycyjnego podejścia w kontekście sygnałów implicit . . 14
2.3. Implicit Alternating Least Squares (iALS) 15
2.3.1. Struktura preferencji i funkcja celu 15
2.3.2. Optymalizacja metodą naprzemiennych najmniejszych kwadratów 16
2.4. Bayesian Personalized Ranking (BPR) 18
2.4.1. Podejście oparte na porównywaniu par (pairwise ranking) 18
2.4.2. Funkcja celu modelu BPR 19
2.4.3. Optymalizacja z wykorzystaniem LearnBPR 21
3. Metryki ewaluacyjne **24**
3.1. Ewaluacja rekomendacji Top-N 24
3.2. Metryki trafności rankingu 24
3.2.1. Precyzja i Pełność 25
3.2.2. Znormalizowany Zdyskontowany Zysk Skumulowany 25
3.3. Metryki pokrycia katalogu, popularności i różnorodności 26
3.3.1. Pokrycie katalogu 26
3.3.2. Obciążenie popularnością 26
3.3.3. Różnorodność wewnątrzlistowa 27
4. Dane i analiza eksploracyjna **28**
4.1. Źródło i struktura danych 28
4.2. Dane jako niejawna informacja zwrotna (implicit feedback) 28
4.3. Podstawowe statystyki surowego zbioru 28
4.4. Wpływ przygotowania danych na strukturę zbioru 29
4.5. Rozkład czasu gry i interakcje z zerowym czasem gry 29
4.6. Aktywność użytkowników i popularność gier 30
4.6.1. Aktywność użytkowników 30
4.6.2. Popularność gier 32
4.7. Wnioski z analizy eksploracyjnej 33

5. Metodologia eksperymentalna **34**
5.1. Przygotowanie danych 34
5.2. Porównywane modele 34
5.3. Strojenie hiperparametrów 35
5.4. Finalna ewaluacja testowa 36
5.5. Analiza wrażliwości na α, β i γ 36
5.6. Ograniczenia metodologii 36
6. Wyniki eksperymentów **38**
6.1. Wyniki strojenia hiperparametrów 38
6.2. Analiza funkcji wag pewności 41
6.3. Jakość rekomendacji na zbiorze testowym 43
6.4. Analiza kompromisów jakości rekomendacji 45
6.5. Stabilność wyników 47
7. Wnioski **50**
7.1. Podsumowanie eksperymentów 50
7.2. Odpowiedź na pytania badawcze 50
7.3. Najważniejsze wnioski praktyczne i metodologiczne 51
7.3.1. Wnioski praktyczne 51
7.3.2. Wnioski metodologiczne 51
7.4. Kierunki dalszych badań 52
Bibliografia **54**
Wykaz symboli i skrótów **57**
Spis rysunków **57**
Spis tabel **58**

1. Wstęp

1.1. Motywacja i problem badawczy

1.2. Cel, zakres i pytania badawcze

1.3. Struktura pracy

2. Podstawy teoretyczne systemów rekomendacyjnych

2.1. Zadanie rekomendacji Top-N

Niech $\mathcal{U} = \{u_1, u_2, \ldots, u_m\}$ oznacza zbiór m użytkowników w systemie, a $\mathcal{I} = \{i_1, i_2, \ldots, i_n\}$ zbiór n dostępnych obiektów.

Relacje zachodzące pomiędzy użytkownikami a obiektami reprezentowane są za pomocą macierzy obserwacji $\mathbf{R} \in \mathbb{R}^{m \times n}$. Pojedynczy element tej macierzy, oznaczany jako r_{ui}, reprezentuje zarejestrowaną aktywność użytkownika $u \in \mathcal{U}$ wobec obiektu $i \in \mathcal{I}$. W zależności od specyfiki przetwarzanych danych, r_{ui} może przyjmować wartości z określonej wcześniej skali (np. ocena w skali 1-5) lub wartości nieograniczone (np. liczba zakupionych produktów, czas odtwarzania danego utworu). Istotną w praktyce cechą macierzy $\mathbf{R}$ jest wysoki stopień rzadkości oznaczający odsetek niezaobserwowanych interakcji. Jest to efekt przytłaczającej ilości dostępnych obiektów powodującej, że pojedynczy użytkownik wchodzi w interakcję jedynie z niewielkim ułamkiem całego asortymentu.

Wczesne sformułowania problemu rekomendacyjnego, oparte na danych jawnych (ang. *explicit feedback*), sprowadzały zadanie do predykcji ocen, gdzie celem algorytmu było odwzorowanie znanych wartości macierzy $\mathbf{R}$ [2]. Jakość takich modeli oceniano za pomocą miar błędu ciągłego opartych na odchyleniu przewidywanej wartości od rzeczywistej oceny, takich jak błąd średniokwadratowy:

$$\mathrm{RMSE} = \sqrt{\frac{1}{|\mathcal{K}|} \sum_{(u,i) \in \mathcal{K}} \left(r_{ui} - \mathbf{x}_u^T \mathbf{y}_i\right)^2}, \tag{2.1}$$

lub średni błąd bezwzględny:

$$\mathrm{MAE} = \frac{1}{|\mathcal{K}|} \sum_{(u,i) \in \mathcal{K}} \left| r_{ui} - \mathbf{x}_u^T \mathbf{y}_i \right|, \tag{2.2}$$

gdzie $\mathcal{K}$ oznacza zbiór znanych par (u, i), dla których zarejestrowano obserwacje r_{ui}.

W praktycznych zastosowaniach predykcja dokładnej wartości oceny okazała się jednak niewystarczająca [7]. Z perspektywy użytkownika istotna jest kolejność rekomendowanych obiektów, a nie dokładna wartość $\hat{r}_{ui}$. Dodatkowo niska wartość RMSE i MAE nie gwarantuje wysokiej trafności, ponieważ model może precyzyjnie przewidywać oceny mało atrakcyjnych obiektów, jednocześnie popełniając błędy przy wyznaczaniu najlepiej dopasowanych rekomendacji. Z tego powodu współcześnie problem jest formułowany jako zadanie rekomendacji Top-N. Celem algorytmu jest wygenerowanie dla danego użytkownika u spersonalizowanej oraz uporządkowanej listy N obiektów z zestawu pozycji dotąd przez niego niezaobserwowanych:

$$\text{Top-}N(u) = \left(i_1^{(u)}, i_2^{(u)}, \ldots, i_N^{(u)}\right) \quad \text{gdzie} \quad i_k^{(u)} \in \mathcal{I} \setminus \mathcal{I}_u \tag{2.3}$$

gdzie $\mathcal{I}_u$ oznacza zbiór obiektów, z którymi użytkownik u wszedł w interakcję. Lista ta jest sortowana według malejącej przewidywanej wartości $\hat{r}_{ui}$ będącej miarą użyteczności obiektu i dla użytkownika u. W tak sformułowanym zadaniu ocena jakości algorytmów opiera się na miarach rankingowych opartych na pozycji obiektu na liście, takich jak precyzja na poziomie K (Precision@K), pełność (Recall@K) czy znormalizowany zdyskontowany zysk skumulowany (NDCG@K).

2.2. Klasyczna faktoryzacja macierzy dla danych jawnych

2.2.1. Model Funk SVD i funkcja celu

W niniejszej pracy wszystkie porównywane modele należą do rodziny modeli opartych na czynnikach ukrytych. Wykorzystują one metody faktoryzacji macierzy, które zostały spopularyzowane przez Simona Funka w ramach konkursu *Netflix Prize*. Podejście to zakłada, że relacje pomiędzy użytkownikami i obiektami mogą być modelowane poprzez ich reprezentacje w przestrzeni czynników ukrytych o wymiarze $f \ll \min(n, m)$. W przeciwieństwie do metod opartych na treści, wymagających jawnego zdefiniowania cech opisujących użytkowników oraz obiekty, faktoryzacja macierzy umożliwia automatyczne uczenie reprezentacji na podstawie historii interakcji. Dzięki temu ogranicza zależność od ręcznie projektowanych cech specyficznych dla danej domeny i pozwala stosować jednolity schemat modelowania w różnych dziedzinach. Szczegółowy opis modelu Funk SVD oraz wynikającej z niego funkcji celu przedstawiono na podstawie pracy [13].

Macierz obserwacji $\mathbf{R} \in \mathbb{R}^{m \times n}$ jest aproksymowana poprzez iloczyn dwóch macierzy niskiego rzędu: macierzy reprezentacji użytkowników $\mathbf{X} \in \mathbb{R}^{m \times f}$ oraz macierzy reprezentacji obiektów $\mathbf{Y} \in \mathbb{R}^{n \times f}$. Wiersz $\mathbf{x}_u \in \mathbb{R}^f$ macierzy $\mathbf{X}$ określa stopień zainteresowania użytkownika u poszczególnymi cechami ukrytymi, natomiast wiersz $\mathbf{y}_i \in \mathbb{R}^f$ macierzy $\mathbf{Y}$ opisuje obiekt i pod kątem tych samych cech. W najprostszym wariancie modelu prognozowana wartość oceny $\hat{r}_{ui}$ wyznaczana jest jako iloczyn skalarny odpowiadających im wektorów:

$$\hat{r}_{ui} = \mathbf{x}_u^T \mathbf{y}_i = \sum_{k=1}^{f} x_{uk} y_{ik} \tag{2.4}$$

W praktyce model ten rozszerza się o wyrazy wolne, które odpowiadają za systematyczne odchylenia takie jak ogólny poziom oceniania w systemie, tendencja konkretnego użytkownika do dawania wyższych/niższych ocen oraz średnia ocena danego obiektu:

$$\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{x}_u^T \mathbf{y}_i \tag{2.5}$$

gdzie $\mu \in \mathbb{R}$ oznacza średnią ocenę ze wszystkich obserwacji w zbiorze treningowym, $b_u \in \mathbb{R}$ jest odchyleniem użytkownika u, a $b_i \in \mathbb{R}$ odchyleniem obiektu i od μ. Należy podkreślić, że odchylenia nie są bezpośrednio liczone na podstawie danych treningowych, lecz dobierane przez model.

Parametry modelu są dobierane na podstawie minimalizacji funkcji straty zdefiniowanej na zbiorze zaobserwowanych ocen $\mathcal{K} = \{(u, i) : r_{ui} \neq \text{null}\}$. Jako podstawowe kryterium dopasowania stosuje się błąd średniokwadratowy. Ze względu na wysoki stopień rzadkości macierzy $\mathbf{R}$ oraz dużą liczbę parametrów ($m \times f + n \times f + m + n$), model bezpośrednio optymalizowany pod kątem minimalizacji błędu na danych treningowych wykazuje silną

tendencję do przeuczenia. Aby aby temu zapobiec, do funkcji celu wprowadza się człony regularyzacyjne oparte na normie L_2. Karzą one wysokie wartości współczynników w wektorach $\mathbf{x}_u$ oraz $\mathbf{y}_i$, a także w wyrazach wolnych b_u, b_i. Zapobiega to dominacji pojedynczych cech i zmniejsza wariancję modelu. Ostateczna funkcja celu $\mathcal{L}$ przyjmuje postać:

$$\begin{aligned}\mathcal{L}(\mathbf{X}, \mathbf{Y}, \mathbf{b}) = &\sum_{(u,i)\in\mathcal{K}} \left(r_{ui} - \mu - b_u - b_i - \mathbf{x}_u^T \mathbf{y}_i\right)^2 \\ &+ \lambda \left(\sum_u \|\mathbf{x}_u\|_2^2 + \sum_i \|\mathbf{y}_i\|_2^2 + \sum_u b_u^2 + \sum_i b_i^2\right)\end{aligned} \tag{2.6}$$

gdzie parametr regularyzacji $\lambda \geqslant 0$ kontroluje siłę kary za złożoność modelu. Celem procesu optymalizacji jest znalezienie macierzy $\mathbf{X}$ i $\mathbf{Y}$ oraz wektorów $\mathbf{b}$, które minimalizują wartość $\mathcal{L}$.

2.2.2. Ograniczenia tradycyjnego podejścia w kontekście sygnałów implicit

Tradycyjne metody faktoryzacji macierzy, takie jak Funk SVD, zostały zaprojektowane z myślą o przetwarzaniu informacji jawnych (ang. *explicit feedback*), opartych na ocenach w skali punktowej. W rzeczywistych systemach produkcyjnych stosowanie tego podejścia wiąże się jednak z istotnymi ograniczeniami. Niektórzy użytkownicy rzadko decydują się na pozostawienie opinii, co utrudnia modelowi dobranie dla nich trafnych rekomendacji. Co więcej, wiele platform nie posiada mechanizmu oceniania, co uniemożliwia działanie klasycznych algorytmów. W konsekwencji uzasadnione staje się wykorzystanie modeli opartych na niejawnych sygnałach zwrotnych (ang. *implicit feedback*), takich jak liczba kliknięć, zarejestrowany czas interakcji pomiędzy użytkownikiem a obiektem czy historia zakupów.

Bezpośrednie zastosowanie klasycznych algorytmów faktoryzacji macierzy do danych niejawnych prowadzi do znacznych problemów, wynikających z różnic między typami sygnałów:

1. **Brak bezpośrednich sygnałów negatywnych oraz dwuznaczność braku interakcji:** W przypadku danych jawnych niska ocena jednoznacznie wskazuje na negatywną preferencję użytkownika. Natomiast dane niejawne dostarczają wyłącznie sygnałów pozytywnych. Długie lub częste interakcje świadczą o zainteresowaniu danym obiektem, podczas gdy brak interakcji ($r_{ui} = 0$) lub interakcje o niewielkiej intensywności mają charakter niejednoznaczny. Mogą one wynikać zarówno z braku zainteresowania obiektem, jak i z faktu, że użytkownik nie wiedział o jego istnieniu.

2. **Problem ignorowania wartości zerowych:** Funkcja celu w Funk SVD optymalizowana jest wyłącznie na zbiorze zaobserwowanych ocen $\mathcal{K}$, całkowicie ignorując pozycje niezaobserwowane. Traktowanie wszystkich brakujących wartości jako obserwacji o wartości $r_{ui} = 0$ prowadziłoby do zdominowania procesu uczenia przez niezaobserwowane interakcje, których liczba wielokrotnie przewyższa liczbę rzeczywistych obserwacji ze względu na wysoką rzadkość macierzy $\mathbf{R}$. Jednocześnie przypisanie im takiej samej wagi jak obserwowanym interakcjom byłoby nieuzasadnione, ponieważ brak interakcji nie oznacza jednoznacznie negatywnej preferencji użytkownika. Z kolei całkowite pominięcie tych wartości powoduje, że model nie wykorzystuje informacji zawartej w niezaobserwowanych interakcjach podczas procesu uczenia.

3. **Różnica między preferencją a wartością obserwacji:** W modelu Funk SVD każda obserwowana wartość r_{ui} wpływa na funkcję celu proporcjonalnie do swojej wartości. W przypadku danych niejawnych liczba interakcji nie odzwierciedla jednak bezpośrednio stopnia preferencji użytkownika. Model nie zawiera mechanizmu pozwalającego oddzielić długość interakcji od preferencji, co sprawia, że obserwacje o dużych wartościach dominują proces uczenia.

Powyższe ograniczenia wymusiły opracowanie dedykowanych modeli dla danych niejawnych. Najważniejsze z nich to implicit ALS (ang. *implicit Alternating Least Squares*), wprowadzający model oparty na rozdzieleniu sygnału preferencji oraz poziomu zaufania do obserwacji, oraz model BPR (ang. *Bayesian Personalised Ranking*), który formułuje problem rekomendacji jako zadanie optymalizacji rankingu. Szczegółowy opis wyżej wymienionych modeli znajduje się w następnych rozdziałach

2.3. Implicit Alternating Least Squares (iALS)

2.3.1. Struktura preferencji i funkcja celu

W przeciwieństwie do modeli przeznaczonych dla danych jawnych, algorytm *Implicit Alternating Least Squares* (iALS), zaproponowany przez Hu, Korena i Volinsky'ego [10], rozdziela zarejestrowaną aktywność r_{ui} na dwie zmienne: binarną preferencję p_{ui} oraz ciągłą wagę zaufania c_{ui} (ang. *confidence*).

Oryginalnie preferencję p_{ui} definiuje się jako zmienną binarną informującą o tym, czy użytkownik u wszedł w interakcję z obiektem i:

$$p_{ui} = \begin{cases} 1 & \text{dla } r_{ui} > 0, \\ 0 & \text{dla } r_{ui} = 0. \end{cases} \tag{2.7}$$

W niniejszej pracy przyjęto odmienną definicję dostosowaną do specyfiki zbioru danych platformy Steam. Zmienna p_{ui} przyjmuje wartość 1, jeśli użytkownik u posiada daną grę w swojej bibliotece, a 0 w przeciwnym wypadku. Oznacza to, że gracz posiadający tytuł w bibliotece, lecz nieposiadający dla niego jeszcze zarejestrowanego czasu gry ($r_{ui} = 0$), nadal ma przypisaną preferencję $p_{ui} = 1$.

Ponieważ zmienna p_{ui} nie uwzględnia intensywności interakcji, do modelu wprowadza się zmienną c_{ui} (w dalszej części pracy określaną jako waga pewności lub confidence). Zwiększa ona swoją wartość wraz ze wzrostem r_{ui}, sygnalizując wyższą pewność algorytmu co do prawdziwości przypisanej preferencji. W pracy Hu et al. podstawowa metoda wyznaczania c_{ui} ma postać:

$$c_{ui} = 1 + \alpha r_{ui} \tag{2.8}$$

gdzie α stanowi hiperparametr kontrolujący tempo wzrostu wagi zaufania.

Warto zaznaczyć, że sposób transformacji surowych sygnałów r_{ui} w wagę c_{ui} ma kluczowy wpływ na zachowanie całego systemu. Wprowadzenie modyfikacji tych funkcji lub dodanie nowych hiperparametrów zmienia sposób, w jaki model traktuje intensywność interakcji, co stanowi główny przedmiot rozważań i analiz w niniejszej pracy.

Model iALS wyznacza wektory cech ukrytych $\mathbf{x}_u \in \mathbb{R}^f$ oraz $\mathbf{y}_i \in \mathbb{R}^f$ poprzez minimalizację ważonej funkcji straty kwadratowej. W przeciwieństwie do modelu Funk SVD, który optymalizuje parametry jedynie na podstawie obserwowanych ocen, implicit ALS uwzględnia wszystkie pary użytkownik-obiekt obejmujące zarówno zaobserwowane, jak i niezaobserwowane interakcje. Jednocześnie każdej parze przypisywana jest waga pewności c_{ui}, która określa znaczenie danej obserwacji podczas procesu uczenia.

Formalnie funkcja celu implicit ALS przyjmuje postać:

$$\mathcal{L}_{\text{iALS}}(\mathbf{X}, \mathbf{Y}) = \sum_{(u,i)} c_{ui} \left(p_{ui} - \mathbf{x}_u^T \mathbf{y}_i\right)^2 + \lambda \left(\sum_{u=1}^{m} \|\mathbf{x}_u\|_2^2 + \sum_{i=1}^{n} \|\mathbf{y}_i\|_2^2 \right), \tag{2.9}$$

gdzie λ jest parametrem odpowiedzialnym za regularyzację.

2.3.2. Optymalizacja metodą naprzemiennych najmniejszych kwadratów

Ze względu na występowanie $m \times n$ par użytkownik-obiekt w funkcji straty modelu iALS, minimalizacja funkcji straty stanowi duże wyzwanie obliczeniowe. Zastosowanie stochastycznego spadku po gradiencie jest w tym przypadku niewydajne, gdyż uwzględnienie wszystkich niezaobserwowanych interakcji wymagałoby przeglądania pełnej macierzy dla każdej epoki. Z tego powodu stosuje się naprzemienne najmniejsze kwadraty jako standardową metodę optymalizacji.

Funkcja celu $\mathcal{L}_{\text{iALS}}(\mathbf{X}, \mathbf{Y})$ nie jest globalnie wypukła względem obu macierzy $\mathbf{X}$ oraz $\mathbf{Y}$ jednocześnie. Sytuacja ulega jednak zmianie, gdy jedną z macierzy traktujemy jako stałą. Gdy macierz $\mathbf{Y}$ traktowana jest jako stała, funkcja straty staje się kwadratowa względem każdego wektora reprezentacji użytkownika $\mathbf{x}_u$ niezależnie, zatem jest również wypukła. Analogicznie, przy ustalonej macierzy $\mathbf{X}$, funkcja staje się wypukła względem każdego wektora reprezentacji obiektu $\mathbf{y}_i$. Własność ta umożliwia analityczne rozwiązanie problemu jako ciągu niezależnych problemów najmniejszych kwadratów.

Algorytm działa iteracyjnie, naprzemiennie wyznaczając optymalne reprezentacje użytkowników przy stałych reprezentacjach obiektów, a następnie optymalne reprezentacje obiektów przy stałych reprezentacjach użytkowników. Aby wyznaczyć wzór aktualizacji dla konkretnego użytkownika u, wyznacza się gradient części funkcji straty zależnej od wektora $\mathbf{x}_u$:

$$\mathcal{L}(u) = \sum_{i=1}^{n} c_{ui} \left(p_{ui} - \mathbf{x}_u^T \mathbf{y}_i\right)^2 + \lambda \|\mathbf{x}_u\|_2^2. \tag{2.10}$$

Różniczkując powyższe wyrażenie względem wektora $\mathbf{x}_u$, otrzymuje się gradient:

$$\frac{\partial \mathcal{L}(u)}{\partial \mathbf{x}_u} = -2 \sum_{i=1}^{n} c_{ui} \left(p_{ui} - \mathbf{x}_u^T \mathbf{y}_i\right) \mathbf{y}_i + 2\lambda \mathbf{x}_u. \tag{2.11}$$

Po zapisaniu powyższego wyrażenia w postaci macierzowej przyjmuje ono formę:

$$\frac{\partial \mathcal{L}(u)}{\partial \mathbf{x}_u} = -2\mathbf{Y}^T \mathbf{C}^u \left(\mathbf{p}(u) - \mathbf{Y}\mathbf{x}_u\right) + 2\lambda \mathbf{x}_u, \tag{2.12}$$

gdzie $\mathbf{C}^u \in \mathbb{R}^{n \times n}$ jest macierzą diagonalną, w której element (i, i) wynosi c_{ui}, a $\mathbf{p}(u) \in \mathbb{R}^n$ jest wektorem binarnych preferencji użytkownika u. Przyrównując tak wyznaczony gradient do zera:

$$-2\mathbf{Y}^T \mathbf{C}^u \mathbf{p}(u) + 2\mathbf{Y}^T \mathbf{C}^u \mathbf{Y}\mathbf{x}_u + 2\lambda \mathbf{x}_u = 0 \tag{2.13}$$

oraz grupując wyrażenia względem $\mathbf{x}_u$:

$$\left(\mathbf{Y}^T\mathbf{C}^u\mathbf{Y} + \lambda\mathbf{I}\right)\mathbf{x}_u = \mathbf{Y}^T\mathbf{C}^u\mathbf{p}(u), \tag{2.14}$$

uzyskuje się analityczne rozwiązanie na optymalny wektor $\mathbf{x}_u$:

$$\mathbf{x}_u = \left(\mathbf{Y}^T\mathbf{C}^u\mathbf{Y} + \lambda\mathbf{I}\right)^{-1}\mathbf{Y}^T\mathbf{C}^u\mathbf{p}(u), \tag{2.15}$$

gdzie $\mathbf{I} \in \mathbb{R}^{f\times f}$ oznacza macierz jednostkową.

Przeprowadzając analogiczne wyprowadzenie dla obiektu i, wyznacza się gradient części funkcji straty zależnej od wektora $\mathbf{y}_i$:

$$\frac{\partial\mathcal{L}(i)}{\partial\mathbf{y}_i} = -2\mathbf{X}^T\mathbf{C}^i\left(\mathbf{p}(i) - \mathbf{X}\mathbf{y}_i\right) + 2\lambda\mathbf{y}_i, \tag{2.16}$$

gdzie $\mathbf{C}^i \in \mathbb{R}^{m\times m}$ jest macierzą diagonalną z wartościami c_{ui} na przekątnej, a $\mathbf{p}(i) \in \mathbb{R}^m$ reprezentuje preferencje wszystkich użytkowników wobec obiektu i. Przyrównanie tego gradientu do zera daje wzór aktualizacji wektora $\mathbf{y}_i$:

$$\mathbf{y}_i = \left(\mathbf{X}^T\mathbf{C}^i\mathbf{X} + \lambda\mathbf{I}\right)^{-1}\mathbf{X}^T\mathbf{C}^i\mathbf{p}(i). \tag{2.17}$$

Ze względu na to, że każdy krok naprzemiennej optymalizacji stanowi dokładne rozwiązanie wypukłego podproblemu, funkcja straty $\mathcal{L}_{\text{iALS}}$ charakteryzuje się własnością monotonicznego spadku w każdej iteracji:

$$\mathcal{L}_{\text{iALS}}(\mathbf{X}^{(t+1)}, \mathbf{Y}^{(t+1)}) \leqslant \mathcal{L}_{\text{iALS}}(\mathbf{X}^{(t+1)}, \mathbf{Y}^{(t)}) \leqslant \mathcal{L}_{\text{iALS}}(\mathbf{X}^{(t)}, \mathbf{Y}^{(t)}). \tag{2.18}$$

Gwarantuje to zbieżność algorytmu do lokalnego minimum.

Obliczenie składnika $\mathbf{Y}^T\mathbf{C}^u\mathbf{Y}$ podczas aktualizacji $\mathbf{x}_u$ wymagałoby czasu $\mathcal{O}(n\cdot f^2)$ dla każdego użytkownika, co prowadziłoby do nieakceptowalnego kosztu obliczeniowego dla dużych zbiorów danych. Można jednak wykorzystać fakt, że macierz pewności może zostać zapisana jako:

$$\mathbf{C}^u = \mathbf{I} + (\mathbf{C}^u - \mathbf{I}),$$

co pozwala przekształcić iloczyn do postaci:

$$\mathbf{Y}^T\mathbf{C}^u\mathbf{Y} = \mathbf{Y}^T\mathbf{Y} + \mathbf{Y}^T(\mathbf{C}^u - \mathbf{I})\mathbf{Y}. \tag{2.19}$$

Pierwszy składnik $\mathbf{Y}^T\mathbf{Y}$ jest niezależny od użytkownika, dlatego może zostać obliczony jednokrotnie na początku iteracji w czasie $\mathcal{O}(n\cdot f^2)$. Z kolei drugi składnik uwzględnia jedynie te pary użytkownik–obiekt, dla których wartość pewności różni się od wartości domyślnej ($c_{ui} \neq 1$). Definiując zbiór takich obserwacji jako:

$$\mathcal{K}_{\text{conf}} = \{(u,i) : c_{ui} \neq 1\},$$

macierz $(\mathbf{C}^u - \mathbf{I})$ pozostaje rzadka, a koszt obliczenia tego drugiego składnika dla wszystkich użytkowników wynosi $\mathcal{O}(|\mathcal{K}_{\text{conf}}|\cdot f^2)$.

Rozwiązanie układu równań (2.14) wymaga wyznaczenia wektora $\mathbf{x}_u$ poprzez rozwiązanie problemu liniowego o wymiarze $f\times f$. Koszt pojedynczej operacji wynosi $\mathcal{O}(f^3)$, a wykonanie jej dla wszystkich m użytkowników prowadzi do składnika $\mathcal{O}(m\cdot f^3)$.

W konsekwencji złożoność aktualizacji całej macierzy użytkowników $\mathbf{X}$ wynosi:

$$\mathcal{O}\left(m\cdot f^3 + |\mathcal{K}_{\text{conf}}|\cdot f^2 + n\cdot f^2\right). \tag{2.20}$$

Analogiczna procedura jest stosowana podczas aktualizacji macierzy obiektów $\mathbf{Y}$. W pełnej iteracji algorytmu iALS oba kroki są wykonywane naprzemiennie, co prowadzi do symetrycznego kosztu względem liczby użytkowników i obiektów.

2.4. Bayesian Personalized Ranking (BPR)

2.4.1. Podejście oparte na porównywaniu par (pairwise ranking)

Alternatywą dla minimalizacji błędu odtwarzania macierzy interakcji stosowanej w modelu iALS są metody oparte na maksymalizacji prawdopodobieństwa a posteriori. Szczególnym przypadkiem takich metod, dobrze dopasowanym do problemu rekomendacji z binarną i ukrytą informacją zwrotną, jest algorytm Bayesian Personalized Ranking (BPR) zaproponowany przez Rendle'a i in. [18]. W przeciwieństwie do modeli przewidujących wartość dopasowania dla pojedynczej pary użytkownik-obiekt, BPR formułuje zadanie rekomendacji jako problem porządkowania parowego (*pairwise ranking*), w którym model uczy się prawdopodobieństwa, że użytkownik preferuje zaobserwowany obiekt i względem niezaobserwowanego obiektu j. Tym samym brak interakcji nie jest bezpośrednio interpretowany jako negatywny sygnał, lecz wykorzystywany do wyznaczania relacji preferencji pomiędzy parami obiektów.

Dla każdego użytkownika $u \in U$ tworzy się trójki danych postaci (u, i, j), gdzie $i \in I_u^+$ stanowi obiekt, z którym użytkownik wszedł w interakcję, natomiast $j \in I \backslash I_u^+$ jest obiektem bez zaobserwowanej interakcji. Relację tę zapisuje się symbolicznie jako $i >_u j$, co oznacza, że użytkownik u preferuje obiekt i nad obiektem j. Zbiór wszystkich tak skonstruowanych trójek w danych treningowych definiuje się jako D_S:

$$D_S = \{(u, i, j) \mid u \in U \wedge i \in I_u^+ \wedge j \in I \setminus I_u^+\}. \tag{2.21}$$

Dla par obiektów, z którymi użytkownik nie wszedł w interakcję (lub wszedł w interakcję z obydwoma), system nie posiada informacji o ich względnej preferencji.

Oznaczmy $>_u$ jako ukryte uporządkowanie zbioru wszystkich obiektów I dla użytkownika u. Relacja ta reprezentuje teoretycznie poprawny, rzeczywisty porządek preferencji użytkownika, który jednak nie jest bezpośrednio obserwowany w danych. Aby relacja ta stanowiła poprawny schemat rankingowy, musi spełniać własności relacji porządku całkowitego:

- **Spójność:** $\forall_{i \neq j \in I} \quad i >_u j \vee j >_u i$,
- **Antysymetria:** $\forall_{i,j \in I} \quad i >_u j \implies \neg(j >_u i)$,
- **Przechodniość:** $\forall_{i,j,k \in I} \quad (i >_u j \wedge j >_u k) \implies i >_u k$.

Do modelowania tak zdefiniowanej relacji porządku wykorzystuje się faktoryzację macierzy. Wynik dopasowania obiektu do użytkownika wyznaczany jest jako iloczyn skalarny odpowiadających im wektorów cech ukrytych $\mathbf{x}_u, \mathbf{y}_i \in \mathbb{R}^f$. Różnica wyników predykcji dla pary obiektów i oraz j przybiera postać:

$$\hat{s}_{uij} = \hat{s}_{ui} - \hat{s}_{uj} = \mathbf{x}_u^T \mathbf{y}_i - \mathbf{x}_u^T \mathbf{y}_j = \mathbf{x}_u^T (\mathbf{y}_i - \mathbf{y}_j). \tag{2.22}$$

Wszystkie uczone parametry modelu zawarte w macierzach cech ukrytych użytkowników $\mathbf{X}$ oraz obiektów $\mathbf{Y}$ oznacza się zbiorczo jako wektor parametrów:

$$\Theta = \mathrm{vec}(\mathbf{X}, \mathbf{Y}), \tag{2.23}$$

gdzie operator vec$(\cdot)$ oznacza przekształcenie wszystkich jej elementów do jednego wektora parametrów.

Aby przekształcić ciągłą różnicę predykcji $\hat{s}_{uij} \in (-\infty, +\infty)$ na prawdopodobieństwo tego, że użytkownik u preferuje obiekt i nad j, stosuje się funkcję sigmoidalną $\sigma(z) = \frac{1}{1+e^{-z}}$:

$$p(i >_u j \mid \Theta) = \sigma(\hat{s}_{uij}) = \frac{1}{1 + e^{-\hat{s}_{uij}}}. \tag{2.24}$$

Warto zauważyć, że zastosowanie funkcji sigmoidalnej $\sigma(z)$ nie tylko sprowadza wartość różnicy predykcji do przedziału $(0, 1)$, ale również gwarantuje zachowanie właściwości antysymetrii relacji preferencji. Ponieważ różnica predykcji jest antysymetryczna ($\hat{s}_{uji} = -\hat{s}_{uij}$), a dla funkcji sigmoidalnej zachodzi tożsamość $1 - \sigma(z) = \sigma(-z)$, model spełnia regułę dopełnienia zdarzeń przeciwnych:

$$p(j >_u i \mid \Theta) = \sigma(\hat{s}_{uji}) = \sigma(-\hat{s}_{uij}) = 1 - \sigma(\hat{s}_{uij}) = 1 - p(i >_u j \mid \Theta). \tag{2.25}$$

2.4.2. Funkcja celu modelu BPR

Celem uczenia modelu Bayesian Personalized Ranking jest wyznaczenie zestawu parametrów Θ, który maksymalizuje prawdopodobieństwo a posteriori zgodne z zaobserwowanymi w danych preferencjami parowymi. Problem ten formułuje się na podstawie twierdzenia Bayesa jako poszukiwanie maksimum:

$$p(\Theta \mid D_S) = \frac{p(D_S \mid \Theta) \cdot p(\Theta)}{p(D_S)}, \tag{2.26}$$

gdzie $p(D_S \mid \Theta)$ stanowi funkcję wiarogodności zbioru obserwacji przy danych parametrach, $p(\Theta)$ jest rozkładem apriori parametrów. Ponieważ mianownik nie zależy od optymalizowanych parametrów, problem maksymalizacji prawdopodobieństwa a posteriori sprowadza się do maksymalizacji licznika:

$$p(\Theta \mid D_S) \propto p(D_S \mid \Theta) \cdot p(\Theta). \tag{2.27}$$

Ze względów stabilności obliczeniowej dokonuje się transformacji powyższego kryterium za pomocą logarytmu naturalnego. Ze względu na ścisłą monotoniczność logarytmu nie zmienia położenia ekstremów funkcji, a pozwala zamienić iloczyny prawdopodobieństw na sumy i zdefiniować funkcję celu:

$$\mathcal{L}_{\text{BPR}}(\Theta) = \ln p(\Theta \mid D_S) = \ln p(D_S \mid \Theta) + \ln p(\Theta) + C, \tag{2.28}$$

gdzie C jest nieistotną z punktu widzenia optymalizacji stałą.

Sformułowanie funkcji wiarygodności

Zakładamy, że wszyscy użytkownicy działają niezależnie od siebie oraz że porządek dla każdej pary obiektów (i, j) dla danego użytkownika jest niezależny od porządku dla pozostałych par. Przyjmując założenie o niezależności wszystkich możliwych relacji w przestrzeni $U \times I \times I$, łączną funkcję wiarygodności ukrytej struktury preferencji można wyrazić jako iloczyn gęstości prawdopodobieństw:

$$p(>|\,\Theta) = \prod_{u \in U} p(>_u|\,\Theta) = \prod_{(u,i,j) \in U \times I \times I} p(i >_u j \mid \Theta)^{\mathbf{1}(i >_u j)} \cdot (1 - p(i >_u j \mid \Theta))^{\mathbf{1}(\neg(i >_u j))}. \tag{2.29}$$

Powyższy wzór można znacząco uprościć, rozpatrując go względem ukrytej struktury preferencji użytkowników. W modelu BPR zakłada się, że ukryta relacja preferencji $>_u$ jest całkowita i antysymetryczna oraz określa rzeczywisty, lecz nieobserwowany porządek obiektów dla każdego użytkownika. Z własności ukrytej relacji preferencji wynikają dwa wnioski:

1. $\neg(i >_u j) \iff j >_u i$.
2. $1 - p(i >_u j \mid \Theta) = p(j >_u i \mid \Theta)$.

Dzięki temu drugi czynnik równania, opisujący relacje przeciwne w ukrytej strukturze preferencji, można przekształcić poprzez zmianę indeksowania:

$$\prod_{\neg(i >_u j)} (1 - p(i >_u j \mid \Theta)) = \prod_{j >_u i} (1 - p(i >_u j \mid \Theta)) = \prod_{j >_u i} p(j >_u i \mid \Theta). \tag{2.30}$$

Ponieważ ukryta relacja preferencji jest całkowita, informacje o relacjach przeciwnych wynikają bezpośrednio z tej samej struktury rankingowej. Oznacza to, że nie stanowią one dodatkowych niezależnych obserwacji, lecz są jednoznacznie określone przez istniejący porządek preferencji użytkownika. W praktyce dostępny jest jedynie obserwowany fragment tej ukrytej relacji zawarty w zbiorze D_S. Zbiór ten zawiera wyłącznie te relacje preferencji, które zostały ujawnione w danych treningowych, dlatego funkcję wiarygodności ogranicza się do zaobserwowanych preferencji:

$$p(D_S \mid \Theta) = \prod_{(u,i,j) \in D_S} p(i >_u j \mid \Theta) = \prod_{(u,i,j) \in D_S} \sigma(\hat{s}_{uij}) \tag{2.31}$$

Po zastosowaniu logarytmu funkcja wiarygodności przyjmuje postać:

$$\ln p(D_S \mid \Theta) = \sum_{(u,i,j) \in D_S} \ln \sigma(\hat{s}_{uij}), \tag{2.32}$$

co odpowiada jednemu z członów w funkcji celu maksymalizowanej w modelu BPR.

Sformułowanie rozkładu apriori i regularyzacji

W celu zdefiniowania rozkładu a priori parametrów modelu zakłada się, że wektor wszystkich parametrów Θ posiada wielowymiarowy rozkład normalny:

$$p(\Theta) = \mathcal{N}(\mathbf{0}, \sigma_\Theta^2 \mathbf{I}), \tag{2.33}$$

Przy założeniu niezależności poszczególnych zmiennych losowych tworzących wektory cech ukrytych $\mathbf{x}_u$ oraz $\mathbf{y}_i$, gęstość wyraża się jako iloczyn jednowymiarowych rozkładów Gaussa:

$$p(\Theta) = \prod_{u \in U} \left(\frac{1}{\sqrt{2\pi\sigma_\Theta^2}^f} \exp\left(-\frac{\|\mathbf{x}_u\|_2^2}{2\sigma_\Theta^2} \right) \right) \cdot \prod_{i \in I} \left(\frac{1}{\sqrt{2\pi\sigma_\Theta^2}^f} \exp\left(-\frac{\|\mathbf{y}_i\|_2^2}{2\sigma_\Theta^2} \right) \right). \tag{2.34}$$

Po nałożeniu logarytmu naturalnego i uporządkowaniu składników otrzymujemy:

$$\ln p(\Theta) = \ln C_\Theta - \frac{1}{2\sigma_\Theta^2} \left(\sum_{u \in U} \|\mathbf{x}_u\|_2^2 + \sum_{i \in I} \|\mathbf{y}_i\|_2^2 \right), \tag{2.35}$$

gdzie C_Θ jest stałą. Przyjmując oznaczenie $\lambda_\Theta = \frac{1}{2\sigma_\Theta^2}$ jako hiperparametr modelu oraz pomijając stałą otrzymujemy drugi człon funkcji celu odpowiedzialny za regularyzację L_2.

$$\ln p(\Theta) \equiv -\lambda_\Theta \left(\sum_{u \in U} \|\mathbf{x}_u\|_2^2 + \sum_{i \in I} \|\mathbf{y}_i\|_2^2 \right). \tag{2.36}$$

Końcowa postać kryterium

Połączenie logarytmu funkcji wiarygodności oraz logarytmu rozkładu aprioti prowadzi do końcowej postaci kryterium optymalizacyjnego, które podlega maksymalizacji:

$$\mathcal{L}_{\text{BPR}}(\mathbf{X}, \mathbf{Y}) = \sum_{(u,i,j) \in D_S} \ln \sigma(\hat{s}_{uij}) - \lambda_\Theta \left(\sum_{u \in U} \|\mathbf{x}_u\|_2^2 + \sum_{i \in I} \|\mathbf{y}_i\|_2^2 \right). \tag{2.37}$$

Uwzględniając postać predykcji z faktoryzacji macierzy ($\hat{s}_{uij} = \mathbf{x}_u^T \mathbf{y}_i - \mathbf{x}_u^T \mathbf{y}_j$), kryterium przybiera pełną postać:

$$\mathcal{L}_{\text{BPR}}(\mathbf{X}, \mathbf{Y}) = \sum_{(u,i,j) \in D_S} \ln \sigma\left(\mathbf{x}_u^T \mathbf{y}_i - \mathbf{x}_u^T \mathbf{y}_j \right) - \lambda_\Theta \left(\sum_{u \in U} \|\mathbf{x}_u\|_2^2 + \sum_{i \in I} \|\mathbf{y}_i\|_2^2 \right). \tag{2.38}$$

2.4.3. Optymalizacja z wykorzystaniem LearnBPR

Wyprowadzone kryterium $\mathcal{L}_{\text{BPR}}(\mathbf{X}, \mathbf{Y})$ nie posiada rozwiązania analitycznego, dlatego jego maksimum wyznacza się metodą iteracyjną, opartą na gradiencie funkcji celu. Naturalnym punktem wyjścia wydaje się algorytm wznoszenia gradientu, w którym w każdej iteracji t aktualizuje się wektor parametrów Θ zgodnie z regułą:

$$\Theta_{t+1} = \Theta_t + \alpha \nabla_\Theta \mathcal{L}_{\text{BPR}}(\Theta_t), \tag{2.39}$$

gdzie α oznacza współczynnik uczenia. Zastosowanie tej reguły w postaci pełnej wymaga jednak wyznaczenia gradientu sumy po wszystkich elementach zbioru treningowego w każdej iteracji algorytmu. Problem ten jest w praktyce czasochłonny dla realistycznych zbiorów danych. Sam zbiór D_S zdefiniowany jest jako iloczyn kartezjański par pozytywnych i negatywnych dla każdego użytkownika, a więc jego liczność wynosi:

$$|D_S| = \sum_{u \in U} |I_u^+| \cdot \left(|I| - |I_u^+| \right), \tag{2.40}$$

co oznacza, że dla typowego użytkownika liczba trójek treningowych jest w przybliżeniu liniowa względem całego katalogu obiektów I. Nawet dla umiarkowanej liczby użytkowników i obiektów zbiór D_S osiąga rozmiary rzędu milionów lub miliardów trójek, co czyni wyznaczanie pełnego gradientu w każdej iteracji zbyt kosztownym obliczeniowo.

Z tego powodu w pracy Rendle i in. [18] zaproponowano algorytm LearnBPR, oparty na stochastycznym wznoszeniu gradientu (ang. *Stochastic Gradient Ascent*) z zastosowaniem próbkowania bootstrapowego. Zamiast obliczać pełny gradient dla całego zbioru D_S, w każdej iteracji algorytmu losowana jest pojedyncza trójka danych (u, i, j). Procedura ta polega na wyborze z równym prawdopodobieństwem użytkownika u, powiązanego z nim obiektu pozytywnego $i \in I_u^+$ oraz obiektu negatywnego $j \in I \setminus I_u^+$. Podejście to pozwala na drastyczne obniżenie kosztu obliczeniowego pojedynczej iteracji, przy jednoczesnym zapewnieniu zbieżności modelu dzięki losowej eksploracji przestrzeni preferencji. Warto jednak zaznaczyć, że w oryginalnym sformułowaniu algorytmu LearnBPR losowanie trójek odbywa się ze zwracaniem i z rozkładu jednostajnego, a nie poprzez sekwencyjne przejście po użytkownikach lub obiektach, jak ma to miejsce w klasycznym stochastycznym wznoszeniu gradientu.

Wyprowadzenie reguł aktualizacji

Rozważmy pojedynczy składnik funkcji celu odpowiadający wylosowanej trójce (u, i, j), ograniczony do występujących w niej parametrów:

$$l(u, i, j) = \ln \sigma(\hat{s}_{uij}) - \lambda_\Theta \left(\|\mathbf{x}_u\|_2^2 + \|\mathbf{y}_i\|_2^2 + \|\mathbf{y}_j\|_2^2 \right). \tag{2.41}$$

Ponieważ $\sigma(z) = \frac{1}{1+e^{-z}}$, a jej pochodna spełnia tożsamość $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, pochodna logarytmu sigmoidy względem argumentu z wynosi:

$$\frac{d}{dz} \ln \sigma(z) = \frac{\sigma'(z)}{\sigma(z)} = 1 - \sigma(z) = \sigma(-z). \tag{2.42}$$

Podstawiając $z = \hat{s}_{uij}$, otrzymujemy:

$$\frac{\partial}{\partial \hat{s}_{uij}} \ln \sigma(\hat{s}_{uij}) = \sigma(-\hat{s}_{uij}). \tag{2.43}$$

Aby wyznaczyć gradienty względem poszczególnych wektorów cech ukrytych, korzystamy z reguły łańcuchowej oraz z jawnej postaci $\hat{s}_{uij} = \mathbf{x}_u^T \mathbf{y}_i - \mathbf{x}_u^T \mathbf{y}_j$, która jest funkcją liniową względem każdego z wektorów $\mathbf{x}_u$, $\mathbf{y}_i$, $\mathbf{y}_j$ z osobna:

$$\nabla_{\mathbf{x}_u} \hat{s}_{uij} = \mathbf{y}_i - \mathbf{y}_j, \qquad \nabla_{\mathbf{y}_i} \hat{s}_{uij} = \mathbf{x}_u, \qquad \nabla_{\mathbf{y}_j} \hat{s}_{uij} = -\mathbf{x}_u. \tag{2.44}$$

Łącząc (2.43) i (2.44) z pochodną członu regularyzacyjnego ($\nabla_{\mathbf{v}} \|\mathbf{v}\|_2^2 = 2\mathbf{v}$), otrzymujemy pełne gradienty pojedynczego składnika $l(u, i, j)$ względem wszystkich trzech występujących w nim parametrów:

$$\nabla_{\mathbf{x}_u} l(u, i, j) = \sigma(-\hat{s}_{uij})(\mathbf{y}_i - \mathbf{y}_j) - 2\lambda_\Theta \mathbf{x}_u, \tag{2.45}$$

$$\nabla_{\mathbf{y}_i} l(u, i, j) = \sigma(-\hat{s}_{uij})\, \mathbf{x}_u - 2\lambda_\Theta \mathbf{y}_i, \tag{2.46}$$

$$\nabla_{\mathbf{y}_j} l(u, i, j) = -\sigma(-\hat{s}_{uij})\, \mathbf{x}_u - 2\lambda_\Theta \mathbf{y}_j. \tag{2.47}$$

Reguły aktualizacji algorytmu LearnBPR

Na podstawie (2.45)-(2.47) oraz reguły gradientu wznoszącego dla pojedynczej próbki, algorytm LearnBPR w każdej iteracji losuje ze zwracaniem trójkę $(u, i, j) \sim D_S$ i wykonuje aktualizację trzech wektorów cech ukrytych:

$$\mathbf{x}_u \leftarrow \mathbf{x}_u + \alpha\Big(\sigma(-\hat{s}_{uij})(\mathbf{y}_i - \mathbf{y}_j) - 2\lambda_\Theta \mathbf{x}_u\Big), \tag{2.48}$$

$$\mathbf{y}_i \leftarrow \mathbf{y}_i + \alpha\Big(\sigma(-\hat{s}_{uij})\,\mathbf{x}_u - 2\lambda_\Theta \mathbf{y}_i\Big), \tag{2.49}$$

$$\mathbf{y}_j \leftarrow \mathbf{y}_j + \alpha\Big(-\sigma(-\hat{s}_{uij})\,\mathbf{x}_u - 2\lambda_\Theta \mathbf{y}_j\Big). \tag{2.50}$$

Procedurę tę powtarza się iteracyjnie aż do spełnienia kryterium zatrzymania (np. ustalonej liczby iteracji lub stabilizacji wartości funkcji celu na zbiorze walidacyjnym). W odróżnieniu od pełnego gradientu wznoszącego, każda iteracja LearnBPR wymaga jedynie stałego, niezależnego od $|D_S|$ nakładu obliczeniowego proporcjonalnego do wymiaru przestrzeni cech ukrytych f. Czyni to tę metodę praktyczną w zastosowaniach dla dużych zbiorów danych.

3. Metryki ewaluacyjne

3.1. Ewaluacja rekomendacji Top-N

Ocena jakości systemów rekomendacyjnych wymaga zastosowania odpowiednio dobranych metryk, które oddają zarówno trafność przewidywań, jak i inne cechy, takie jak obciążenie popularnością, pokrycie katalogu czy różnorodność. Niniejszy rozdział przedstawia metryki ewaluacyjne zastosowane w pracy, wraz z ich definicją i interpretacją.

Niech $\mathcal{U}$ oznacza zbiór wszystkich użytkowników, a $\mathcal{I}$ zbiór wszystkich obiektów w katalogu. W procesie ewaluacji zbiór interakcji każdego użytkownika $u \in \mathcal{U}$ dzieli się na część treningową oraz testową, co pozwala zdefiniować zbiory:

- $\mathcal{I}_u^{\text{train}}$ – zbiór obiektów, z którymi użytkownik u wszedł w interakcję w zbiorze treningowym,
- $\mathcal{I}_u^{\text{test}}$ – zbiór obiektów, z którymi użytkownik u wszedł w interakcję w zbiorze testowym,

Jak wskazano w sekcji 2.1, współczesne systemy rekomendacyjne rzadko koncentrują się na predykcji ciągłych wartości ocen, a zamiast tego rozwiązują zadanie rekomendacji Top-N oceniane za pomocą metryk rankingowych [14]. Celem algorytmu jest wygenerowanie dla użytkownika u uporządkowanej listy N najbardziej trafnych obiektów ze zbioru pozycji dotąd przez niego niezaobserwowanych:

$$\text{Top-}N(u) = \left(i_1^{(u)}, i_2^{(u)}, \ldots, i_N^{(u)}\right) \quad \text{gdzie} \quad i_j^{(u)} \in \mathcal{I} \setminus \mathcal{I}_u^{\text{train}} \tag{3.1}$$

gdzie parametr N określa rozmiar prezentowanej użytkownikowi listy.

Zbiór obiektów rekomendowanych użytkownikowi u, pozbawiony informacji o ich kolejności na liście, oznacza się jako $\mathcal{R}_u^N = \{i_1^{(u)}, i_2^{(u)}, \ldots, i_N^{(u)}\}$. Podstawowym pojęciem potrzebnym do oceny skuteczności rekomendacji na jest wyznaczenie części wspólnej zbioru pozycji rekomendowanych $\mathcal{R}_u^N$ oraz zbioru pozycji rzeczywiście istotnych w zbiorze testowym $\mathcal{I}_u^{\text{test}}$, co pozwala ustalić liczbę trafnych rekomendacji:

$$\text{Hits}_u@N = \left|\mathcal{R}_u^N \cap \mathcal{I}_u^{\text{test}}\right| \tag{3.2}$$

3.2. Metryki trafności rankingu

Metryki trafności oceniają zdolność systemu rekomendacyjnego do identyfikacji obiektów zawartych w zbiorze testowym. W kontekście rekomendacji Top-N najpopularniejszymi miarami są precyzja (Precision@N), pełność (Recall@N) oraz znormalizowany zdyskontowany zysk skumulowany (NDCG@N) [11, 2].

3.2.1. Precyzja i Pełność

Precyzja na poziomie N określa uśredniony stosunek liczby trafnych rekomendacji do całkowitej liczby obiektów zaprezentowanych na rekomendowanej liście [11]:

$$\text{Precision@}N = \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \frac{\text{Hits}_u@N}{N} = \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \frac{\left| \mathcal{R}_u^N \cap \mathcal{I}_u^{\text{test}} \right|}{N} \tag{3.3}$$

Pełność na poziomie N mierzy natomiast, jaka część obiektów istotnych dla użytkownika zawartych w zbiorze testowym została uwzględniona na wygenerowanej liście Top-N [11]:

$$\text{Recall@}N = \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \frac{\text{Hits}_u@N}{|\mathcal{I}_u^{\text{test}}|} = \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \frac{\left| \mathcal{R}_u^N \cap \mathcal{I}_u^{\text{test}} \right|}{|\mathcal{I}_u^{\text{test}}|} \tag{3.4}$$

Mimo szerokiego zastosowania obie przedstawione metryki posiadają istotne ograniczenia. Precision@N jest silnie uzależniona od liczby dostępnych interakcji testowych użytkownika. W przypadku gdy $|\mathcal{I}_u^{\text{test}}| < N$, maksymalna teoretyczna wartość precyzji dla takiego użytkownika jest mniejsza niż 1 i maleje wraz ze wzrostem długości listy N. Z kolei Recall@N monotonicznie rośnie wraz ze wzrostem długości listy N, co oznacza, że ustawienie zbyt dużego parametru N może prowadzić do sztucznie wysokich wartości pełności.

3.2.2. Znormalizowany Zdyskontowany Zysk Skumulowany

W celu wyeliminowania ograniczeń wyżej opisanych miar stosuje się znormalizowany zdyskontowany zysk skumulowany (NDCG@N) [2]. Metryka ta nagradza wyższe pozycje trafnych obiektów na liście poprzez wprowadzenie logarytmicznego czynnika dyskontującego:

$$\text{DCG}_u@N = \sum_{k=1}^{N} \frac{\mathbf{1}\left(i_k^{(u)} \in \mathcal{I}_u^{\text{test}} \right)}{\log_2(k+1)} \tag{3.5}$$

Aby umożliwić porównywanie wyników pomiędzy użytkownikami o różnej liczbie istotnych obiektów, wartość $\text{DCG}_u@N$ normalizuje się poprzez idealną wartość tej metryki ($\text{IDCG}_u@N$), będącą wynikiem dla optymalnego uporządkowania, w którym wszystkie trafne obiekty znajdują się na najwyższych pozycjach listy:

$$\text{IDCG}_u@N = \sum_{k=1}^{\min(N, |\mathcal{I}_u^{\text{test}}|)} \frac{1}{\log_2(k+1)} \tag{3.6}$$

Ostateczna wartość NDCG@N obliczana jest jako średnia ze znormalizowanych wartości dla wszystkich użytkowników:

$$\text{NDCG@}N = \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \frac{\text{DCG}_u@N}{\text{IDCG}_u@N} \tag{3.7}$$

NDCG@N jest współcześnie jedną z najczęściej stosowanych metryk ewaluacyjnych w analizie rekomendacji rankingowych[21, 12, 23]. Jej kluczową zaletą jest jednoczesne uwzględnienie trafności rekomendacji oraz pozycji obiektu na liście, a także odporność na zróżnicowaną liczbę interakcji testowych poszczególnych użytkowników.

3.3. Metryki pokrycia katalogu, popularności i różnorodności

Maksymalizacja trafności rekomendacji nie jest jedynym kryterium decydującym o sukcesie systemu rekomendacyjnego. Algorytmy optymalizujące metryki rankingowe, mają tendencję do generowania tzw. "bezpiecznych"poleceń. W efekcie system może rekomendować wąski podzbiór katalogu, ograniczony do najpopularniejszych pozycji bądź oferować listę podobnych do siebie pozycji. Przykładowo, przedstawienie użytkownikowi listy złożonej wyłącznie z kolejnych odsłon tej samej serii,mimo wysokiego prawdopodobieństwa ich trafności, ogranicza możliwość odkrywania nowych treści. Z tego powodu ewaluacja systemów rekomendacyjnych wymaga uwzględnienia miar oceniających pokrycie katalogu, obciążenie popularnością oraz różnorodność wygenerowanych rekomendacji.

3.3.1. Pokrycie katalogu

Pokrycie katalogu mierzy odsetek wszystkich dostępnych w systemie obiektów, które zostały zarekomendowane co najmniej jednemu użytkownikowi na liście długości N:

$$\text{Coverage@}N = \frac{\left|\bigcup_{u \in \mathcal{U}} \mathcal{R}_u^N\right|}{|\mathcal{I}|} \tag{3.8}$$

Niska wartość tej metryki wskazuje na zjawisko, w którym znaczna część asortymentu pozostaje całkowicie niewidoczna dla użytkowników. Jest to często obserwowane w implementacjach systemów rekomendacyjnych w danych, gdzie występuje zjawisko długiego ogona, w którym większość interakcji koncentruje się na niewielkiej liczbie popularnych obiektów.

3.3.2. Obciążenie popularnością

Do oceny stopnia obciążenia systemu popularnością stosuje się metrykę ΔGAP (ang. *Delta Group Average Popularity*) [1]. Wyznacza ona względną różnicę pomiędzy średnią popularnością obiektów występujących na wygenerowanych listach rekomendacji Top-N a średnią popularnością obiektów znajdujących się w historycznych profilach użytkowników. Żeby ją zdefiniowac, należy najpierw wprowadzić kilka powiązanych pojęć.

Popularność pojedynczego obiektu $i \in \mathcal{I}$ definiuje się jako stosunek liczby użytkowników, którzy weszli z nim w interakcję w zbiorze treningowym, do całkowitej liczby użytkowników:

$$\text{pop}(i) = \frac{\left|\{u \in \mathcal{U} \mid i \in \mathcal{I}_u^{\text{train}}\}\right|}{|\mathcal{U}|} \tag{3.9}$$

Na podstawie tak zdefiniowanej popularności pojedynczych obiektów wyznacza się średnią popularność rekomendacji (GAP_{rec}) oraz średnią popularność profili użytkowników ($\text{GAP}_{\text{profile}}$):

$$\text{GAP}_{\text{rec}} = \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \frac{1}{N} \sum_{i \in \mathcal{R}_u^N} \text{pop}(i) \tag{3.10}$$

$$\text{GAP}_{\text{profile}} = \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \frac{1}{|\mathcal{I}_u^{\text{train}}|} \sum_{i \in \mathcal{I}_u^{\text{train}}} \text{pop}(i) \tag{3.11}$$

Ostateczna wartość wskaźnika ΔGAP stanowi względną zmianę średniej popularności wygenerowanych rekomendacji w odniesieniu do dotychczasowych preferencji użytkowników:

$$\Delta\text{GAP} = \frac{\text{GAP}_{\text{rec}} - \text{GAP}_{\text{profile}}}{\text{GAP}_{\text{profile}}} \tag{3.12}$$

Warto zaznaczyć, że w oryginalnej pracy [1] wartość ΔGAP obliczana jest osobno dla zdefiniowanego wcześniej podziału użytkowników na grupy. W niniejszej pracy metrykę tę stosuje się w ujęciu globalnym, obliczając ją dla całego zbioru użytkowników $\mathcal{U}$.

Wartość $\Delta\text{GAP} > 0$ oznacza, że średnia popularność rekomendowanych obiektów jest wyższa niż średnia popularność pozycji obecnych w profilach użytkowników, co wskazuje na tendencję systemu do faworyzowania pozycji popularnych. Wartość $\Delta\text{GAP} < 0$ świadczy natomiast o tym, że algorytm częściej rekomenduje pozycje bardziej niszowe niż te występujące dotychczas w historii interakcji.

3.3.3. Różnorodność wewnątrzlistowa

Różnorodność wewnątrzlistowa mierzy stopień zróżnicowania obiektów znajdujących się na pojedynczej liście Top-N. W kontekście modeli opartych na czynnikach ukrytych, zróżnicowanie pomiędzy dwoma obiektami i oraz j definiuje się jako odległość cosinusową ich wektorów cech utajonych $\mathbf{y}_i, \mathbf{y}_j \in \mathbb{R}^f$:

$$d_{\cos}(i,j) = 1 - \frac{\mathbf{y}_i \cdot \mathbf{y}_j}{\|\mathbf{y}_i\|_2 \|\mathbf{y}_j\|_2} \tag{3.13}$$

Średnia różnorodność dla pojedynczego użytkownika u wyznaczana jest jako uśredniona odległość cosinusowa dla wszystkich unikalnych par obiektów znajdujących się na jego liście rekomendacyjnej Top-N:

$$\text{Div}_u@N = \frac{2}{N(N-1)} \sum_{i=1}^{N-1} \sum_{j=i+1}^{N} d_{\cos}\left(i_i^{(u)}, i_j^{(u)}\right) \tag{3.14}$$

Ostateczna wartość metryki Diversity@N stanowi średnią arytmetyczną z wartości różnorodności wyznaczonych dla wszystkich użytkowników:

$$\text{Diversity}@N = \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \text{Div}_u@N \tag{3.15}$$

Wyższa wartość metryki wskazuje na większe zróżnicowanie cech obiektów prezentowanych na pojedynczej liście rekomendacji, co ogranicza ryzyko rekomendowania podobnych obiektów. Należy jednak pamiętać o istotnych ograniczeniach tak zdefiniowanej różnorodności. Wskaźnik ten nie bazuje na rzeczywistych cechach obiektów, lecz na ich reprezentacjach w utajonej przestrzeni czynników, uzyskanych w procesie uczenia. Z tego powodu ta metryka nie pozwala na bezpośrednie porównywanie różnych modeli. Przestrzeń czynników ukrytych każdego z modeli jest konstruowana niezależnie oraz wymiar może różnić się między modelami. Metryka ta stanowi natomiast narzędzie diagnostyczne, pozwalające na wykrywanie degeneracji przestrzeni czynników. Wartości Diversity@N zbliżające się do zera oznaczają, że wektory obiektów uległy koncentracji wokół jednego kierunku w przestrzeni $\mathbb{R}^f$. To zjawisko prowadzi do utraty zdolności modelu do różnicowania obiektów i w konsekwencji do generowania list rekomendacyjnych zdominowanych przez obiekty najbardziej popularne.

4. Dane i analiza eksploracyjna

4.1. Źródło i struktura danych

Dane wykorzystane w pracy pochodzą z publicznie dostępnej kolekcji zbiorów danych zebranych przez zespół Juliana McAuleya [16] z Uniwersytetu Kalifornijskiego w San Diego w celu budowy systemów rekomendacyjnych i personalizacji. W szczególności wykorzystano podzbiór danych zawierający biblioteki użytkowników platformy Steam sprzedającej gry komputerowe.

Źródłowe dane miały postać pliku JSON, w którym każdy rekord opisywał bibliotekę jednego użytkownika, identyfikowanego przez zmienną `user_id`. Dla każdego użytkownika dostępna była lista przypisanych do niego gier wraz z identyfikatorem gry (`item_id`), nazwą (`item_name`) oraz łącznym czasem rozgrywki (`playtime_forever`) w minutach.

Dane wejściowe zostały przekształcone do postaci ramki danych, w której pojedynczy wiersz odpowiada unikalnej parze użytkownik-gra, a następnie do postaci rzadkiej macierzy, której wiersze odpowiadają poszczególnym klientom, kolumny produktom, a wartości dlugości interakcji miedzy poszczególną parą. Jest to standardowa struktura danych w stosowanej w pracy bibliotece systemów rekomendacyjnych.

4.2. Dane jako niejawna informacja zwrotna (implicit feedback)

Wykorzystane dane nie zawierają jawnych ocen przypisanych przez użytkowników, takich jak wartości liczbowe lub gwiazdki. Jedyne sygnały behawioralne informujące o relacji konsumenta z danym produktem jest jego obecność w bibliotece oraz całkowity czas trwania interakcji. Zatem rekomendacje będą oparte na zmiennej `playtime_forever`, która pełni rolę miary intensywności relacji między danym użytkownikiem i produktem. Z tego powodu jest to problem rekomendacji na podstawie niejawnej informacji (implicit feedback). Warto podkreślić, że jest to silnie zaszumiony sygnał i jego wartość nie odzwierciedla proporcjonalnie zainteresowania danym tytułem. Stanowi to motywację do badania wpływu przekształceń tej zmiennej na jakość rekomendacji.

4.3. Podstawowe statystyki surowego zbioru

Przed filtracją analizowany zbiór zawierał 5 153 209 interakcji, obejmujących 70 912 użytkowników oraz 10 978 gier. Wynika stąd, że obserwowana macierz użytkownik-gra była bardzo rzadka: jej gęstość wynosiła około 0,00662, co znaczy, że zaobserwowane zostało zaledwie 0,662% wszystkich możliwych interakcji w macierzy. Niska gęstość macierzy jest powszechna w dziedzinie systemów rekomendacyjnych, ponieważ przeciętny konsument wybierze zaledwie kilka produktów z katalogu zawierającego tysiące lub miliony rekordów.

Tabela 4.1: Podstawowe statystyki surowego zbioru danych

Statystyka	Wartość
Liczba interakcji użytkownik-gra	5 153 209
Liczba użytkowników	70 912
Liczba gier	10 978
Gęstość macierzy	0,00662

4.4. Wpływ przygotowania danych na strukturę zbioru

Przed treningiem i ewaluacją modeli dane zostały poddane filtracji. Szczegółowy opis i uzasadnienie tych decyzji przedstawiono w rozdziale dotyczącym metodologii eksperymentalnej. W tym podrozdziale omówiono jedynie ich wpływ na rozmiar oraz strukturę finalnego zbioru.

Tabela 4.2: Wpływ kolejnych etapów przygotowania danych na rozmiar zbioru

Etap	Interakcje	Użytkownicy	Gry
Dane surowe	5 153 209	70 912	10 978
Po filtracji gier	5 149 512	70 912	9 198
Po filtracji użytkowników	5 132 610	62 944	9 198

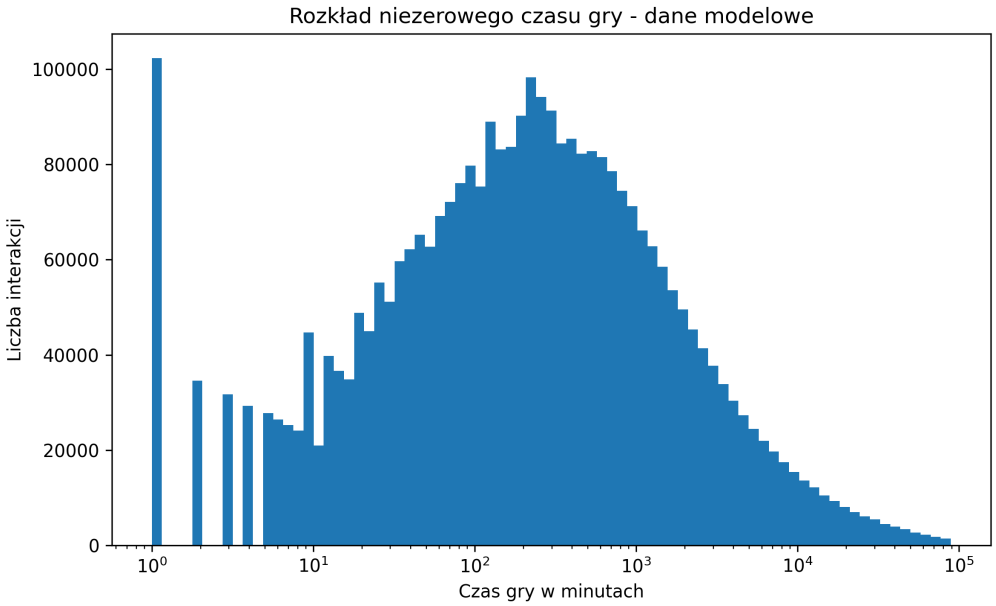
Usunięto 20 599 interakcji, co odpowiada około 0,40% początkowego zbioru. Tymczasem liczba użytkowników zmniejszyła się o 7 968, czyli o 11,24%, a liczba gier o 1 780, czyli o 16,21%. Świadczy to o tym, że przygotowanie danych usunęło przede wszystkim niepopularne gry i mało aktywnych użytkowników, zachowując niemal cały zasób informacji o obserwowanych interakcjach. Gęstość macierzy użytkownik-gra wzrosła z 0,00662 do około 0,00887, jednak nadal pozostała bardzo niska. Zatem zbiór po przygotowaniu nadal ma charakter rzadkich danych rekomendacyjnych, ale jest mniej obciążony przypadkami o małej liczbie interakcji.

4.5. Rozkład czasu gry i interakcje z zerowym czasem gry

Istotną cechą analizowanego zbioru jest występowanie interakcji, dla których wartość zmiennej `playtime_forever` wynosi zero. Oznacza to, że dana gra znajduje się w bibliotece użytkownika, ale nie odnotowano żadnego czasu rozgrywki. Interakcje z zerowym czasem gry stanowiły 36,25% wszystkich obserwacji. Usunięcie ich z analizy zmieniłoby definicję problemu, model uczyłby się wyłącznie na podstawie gier faktycznie uruchomionych, pomijając znaczną liczbę pozytywnych obserwacji. Ponieważ sama obecność gry w bibliotece jest sygnałem behawioralnym świadczącym o zainteresowaniu produktem. W niniejszej pracy obserwacje z `playtime_forever = 0` zostały zachowane jako poprawne interakcje użytkownik-gra. Nie jest to podejście standardowe, gdyż w większości prac poświęconych danym z tej platformy rekordy bez zarejestrowanego czasu gry są usuwane [5, 6]. Świadome odejście od tej praktyki wynika z chęci uniknięcia utraty ponad jednej trzeciej danych oraz

zachowania sygnału intencji zakupu, który z punktu widzenia analizy preferencji konsumenckich wciąż niesie ze sobą istotną wartość informacyjną.

Dla interakcji z dodatnim czasem gry rozkład zmiennej `playtime_forever` jest silnie prawostronnie skośny. Większość obserwacji koncentruje się przy stosunkowo niskich wartościach, a duże czasy rozgrywki tworzą długi ogon. Na rysunku 4.1 przedstawiono histogram dodatnich wartości czasu gry w skali logarytmicznej.



Rysunek 4.1: Rozkład dodatniego czasu gry w skali logarytmicznej w przygotowanym zbiorze

Skośność rozkładu czasu gry uzasadnia użycie funkcji wag pewności o spowolnionym wzroście. Liniowe potraktowanie zmiennej `playtime_forever` nadawałoby bardzo dużą wagę skrajnym obserwacjom, przez co mogłoby zaburzać proces uczenia modelu. Natomiast sam fakt obecności gry w bibliotece pozostaje istotną informacją nawet wtedy, gdy czas gry jest równy zero. Z tego powodu w niniejszej pracy zbadane zostały funkcje wag pewności redukujące wariancję zmiennej odpowiadającej za czas gry oraz nadające obserwacjom z zerowym czasem gry zwiększoną wagę bazową. Definicje poszczególnych funkcji wag pewności znajdują się w sekcji 5.2.

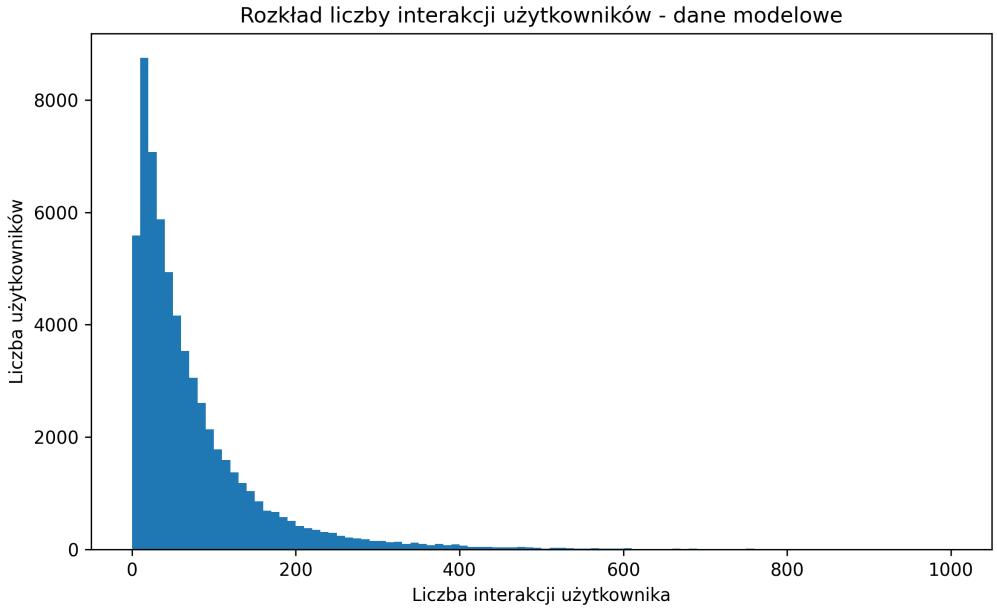
4.6. Aktywność użytkowników i popularność gier

4.6.1. Aktywność użytkowników

Rozkład liczby interakcji przypadających na użytkownika jest prawostronnie skośny. Na rysunku 4.2 przedstawiono histogram liczby interakcji przypadających na użytkownika. Widoczne skupienie przy niskich wartościach oraz długi ogon pokazują, że zbiór łączy dużą liczbę użytkowników o umiarkowanej aktywności z niewielką grupą kont posiadających bardzo rozbudowane biblioteki. W przygotowanym zbiorze średnia liczba interakcji na użytkownika wynosiła 81,54, podczas gdy mediana była równa 48.

Tabela 4.3: Rozkład liczby interakcji użytkowników i popularności gier w przygotowanym zbiorze

Statystyka	Interakcje użytkownika	Interakcje gry
Średnia	81,54	558,01
Mediana	48	71
95. percentyl	251	2 515,05
99. percentyl	518,57	8 811,84
Maksimum	7 337	45 699

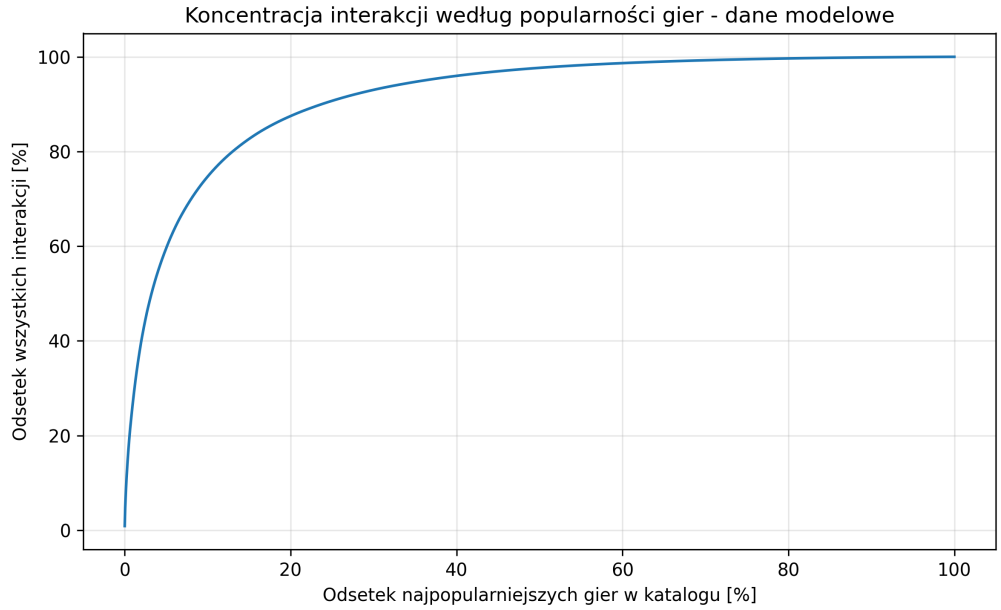


Rysunek 4.2: Rozkład liczby interakcji przypadających na użytkownika w przygotowanym zbiorze danych

4.6.2. Popularność gier

Jeszcze silniejszą nierównomierność widać po stronie gier. Mediana liczby interakcji przypadających na grę wynosiła 71, podczas gdy średnia była równa 558,01. Wynika stąd, że większość gier występowała w bibliotekach stosunkowo małej liczby użytkowników, natomiast niewielka grupa najpopularniejszych tytułów odpowiadała za bardzo dużą część wszystkich interakcji.

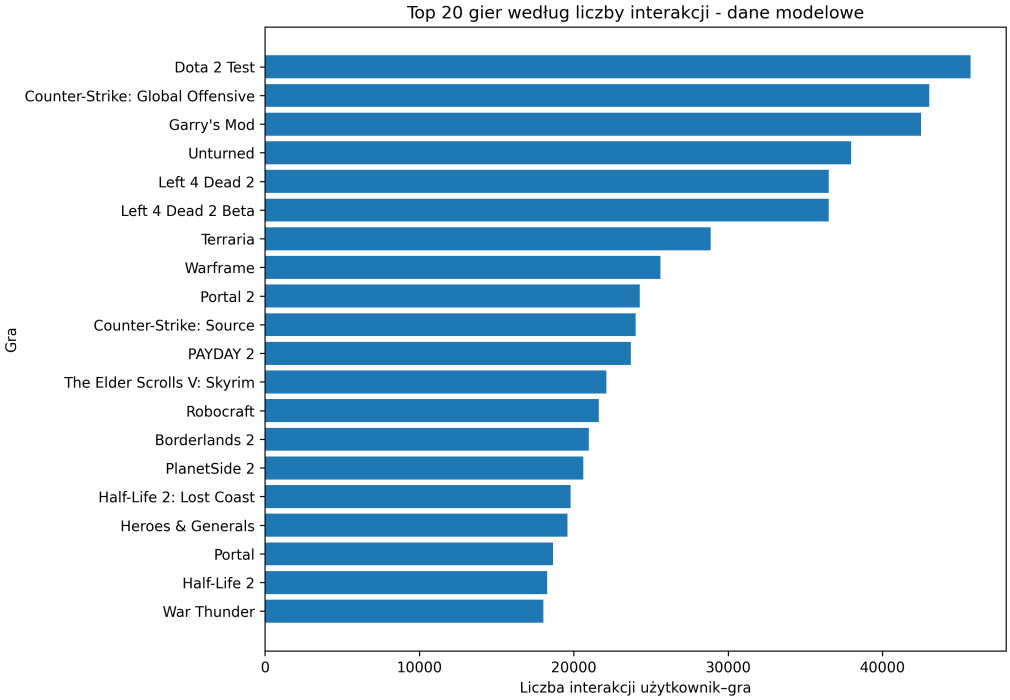
Koncentrację popularności przedstawiono na rysunku 4.3. Najpopularniejszy 1% gier odpowiadał za około 27,16% wszystkich interakcji, najpopularniejsze 5% za 59,55%, a najpopularniejsze 10% za 74,71%.



Rysunek 4.3: Koncentracja interakcji według popularności gier w przygotowanym zbiorze danych

Dodatkową ilustracją tej nierównomierności jest zestawienie najpopularniejszych gier przedstawione na rysunku 4.4. Wśród tytułów o największej liczbie interakcji znajdują się przede wszystkim bardzo popularne gry wieloosobowe oraz darmowe produkcje obecne w bibliotekach dużej liczby użytkowników. Obecność takich pozycji pokazuje, że prosty model oparty wyłącznie na popularności mógłby generować pozornie atrakcyjne rekomendacje, ale jednocześnie silnie ograniczałby różnorodność katalogu.

Zaobserwowana koncentracja popularności ma znaczenie dla interpretacji późniejszych wyników eksperymentalnych. Modele rekomendacyjne mogą osiągać dobrą trafność, często rekomendując najpopularniejsze tytuły, ale takie zachowanie prowadzi do niskiego pokrycia katalogu i silnego obciążenia rekomendacji popularnością. Z tego powodu w dalszej części pracy, obok metryk trafności rankingowej, analizowane są również miary pokrycia katalogu oraz popularności rekomendowanych gier.



Rysunek 4.4: Dwadzieścia najpopularniejszych gier według liczby interakcji w przygotowanym zbiorze danych

4.7. Wnioski z analizy eksploracyjnej

Najważniejsze cechy analizowanego zbioru danych, które bezpośrednio wpłynęły na przyjętą metodologię badawczą, to silnie prawostronnie skośny rozkład czasu gry użytkowników oraz wyraźna koncentracja popularności gier. Rozkład zmiennej `playtime_forever` charakteryzuje się długim ogonem, co wymaga odpowiedniego przekształcenia tej zmiennej w wagę pewności wykorzystywaną w modelu. Z kolei nierównomierny rozkład popularności gier oznacza, że niewielka część katalogu generuje znaczną część interakcji, co należy uwzględnienić w ocenie jakości rekomendacji poprzez analizę miar pokrycia katalogu oraz obciążenia popularnością.

Kolejnym istotnym aspektem jest rola obserwacji z wartością `playtime_forever = 0`, które odmiennie od dominującego w literaturze trendu ich usuwania [5, 6] zostały zachowane w procesie uczenia jako istotny sygnał behawioralny. Ich efektywne wykorzystanie wymaga przekształcenia czasu gry w wagę ufności, która zapewnia tym obserwacjom podwyższony poziom bazowy, a w połączeniu ze spowolnieniem przyrostu wagi dla rosnących czasów gry pozwala na lepsze wykorzystanie informacji zawartych w danych wejściowych.

5. Metodologia eksperymentalna

5.1. Przygotowanie danych

Przed właściwym procesem trenowania modeli dane zostały poddane wstępnej obróbce. W analizowanym zbiorze nie występowały braki danych, więc ich filtracja nie była konieczna. Ze względu na obecność wartości odstających zmiennej `playtime_forever` zdecydowano się na ograniczenie jej wartości do 120000 minut, co odpowiada 2000 godzinom, oraz w przybliżeniu 99,97 percentylowi w zbiorze danych. Obserwacje powyżej tej granicy zostały zastąpione wartością 120000. Przyjęty próg pozwala na wyeliminowanie potencjalnych błędów pomiarowych, przy jednoczesnym zachowaniu sygnału o ekstremalnym zaangażowaniu użytkowników.

Usunięto również gry przypisane do mniej niż 5 użytkowników oraz użytkowników posiadających w swojej bibliotece mniej niż 5 gier. Miało to na celu zmiejszenia wpływu zimnego startu na wyniki modeli oraz możliwość późniejszego podziału danych na zbiory treningowy, walidacyjny i testowy, w sposób zapewniający, że każdy użytkownik pojawi się w każdym z tych zbiorów.

Zbiór po filtracji został podzielony na zbiory: treningowy, walidacyjny i testowy. Dla każdego użytkownika do zbioru testowego przypisano liczbę interakcji równą $\lceil 0{,}1 \cdot n_u \rceil$, gdzie n_u oznacza liczbę interakcji danego użytkownika. Zastosowano zaokrąglenie w górę, aby zapewnić, że każdy użytkownik będzie miał co najmniej jedną interakcję w zbiorze testowym. Podział został przeprowadzony w sposób losowy ze względu na brak inforamcji o datach przypisania produktów do bibliotek użytkowników. Z pozostałej część danych (stanowiącej w przybliżeniu 90%) wydzielono do zbioru walidacyjnego liczbę interakcji wyznaczoną analogicznie, czyli $\lceil 0{,}1 \cdot n_{u,\text{rem}} \rceil$, gdzie $n_{u,\text{rem}}$ oznacza pozostałą po pierwszym podziale liczbę interakcji użytkownika. Ostateczna struktura podziału danych wynosi zatem w przybliżeniu 81% dla zbioru treningowego, 9% dla walidacyjnego oraz 10% dla testowego, co jest rozwiązaniem bardzo zbliżonym do proporcji stosowanych m.in. w pracach [15, 17]. W celu zapewnienia pełnej reprodukowalności wyników, na każdym etapie ustalono stałą wartość ziarna generatora liczb pseudolosowych.

5.2. Porównywane modele

Porównano siedem modeli rekomendacyjnych, w tym pięć modeli implicit ALS z różnymi funkcjami wag pewności (confidence), model Bayesian Personalized Ranking (BPR) zawarte w bibliotece implicit `implicit` [8] oraz model popularnościowy jako punkt odniesienia. Zastosowane funkcje wag pewności w modelach implicit ALS to:

- $c_{ui} = 1 + \alpha r_{ui}$,

- $c_{ui} = 1 + \alpha \log(1 + r_{ui})$,
- $c_{ui} = 1 + \alpha \frac{\log(1+r_{ui})}{1+\log(1+r_{ui})}$,
- $c_{ui} = 1 + \alpha \log(1 + r_{ui} + \beta)$,
- $c_{ui} = 1 + \gamma$,

gdzie r_{ui} oznacza wartość zmiennej `playtime_forever` dla użytkownika u i gry i, a α stanowi skalar regulujący siłę wpływu czasu gry. Dwie pierwsze funkcje stanowią klasyczne sformułowania zaproponowane w pracy [10]. Pozostałe trzy funkcje są autorską propozycją modyfikacji, opracowaną na podstawie analizy rozkładu zmiennej `playtime_forever` oraz wyników eksperymentów.

W szczególności, funkcja $c_{ui} = 1 + \alpha \log(1 + r_{ui} + \beta)$ została zaprojektowana w celu zmniejszenia wpływu szumu wynikającego z dużej liczby obserwacji o niskim czasie gry, przy jednoczesnym wprowadzeniu przesunięcia β. Pozwala ono nadawać obserwacjom z `playtime_forever = 0` wagę ufności większą niż 1, co odzwierciedla wstępne zainteresowanie użytkownika wyrażone samym posiadaniem pozycji w bibiliotece. Aby zweryfikować, czy ewentualna poprawa wyników wynika z połączonego efektu kompresji szumu i dodatkowej wagi bazowej, czy też wyłącznie z faktu przypisania wyższej ufności zerowym obserwacjom, wprowadzono dodatkowo funkcję $c_{ui} = 1 + \gamma$, gdzie γ jest hiperparametrem. W celu uniknięcia arbitralnego wyboru wartości nowych hiperparametrów β oraz γ poddano procesowi optymalizacji.

5.3. Strojenie hiperparametrów

Proces strojenia hiperparametrów został przeprowadzony na podstawie zbioru walidacyjnego. Celem była optymalizacja NDCG@10 dla każdego z modeli. Metryka ta została wybrana ze względu na jej własności opisane w podpodrozdziale 3.2.2. Procedura miała charakter dwuetapowy. Najpierw przeprowadzono szeroki grid search dla wszystkich modeli, aby znaleźć przybliżony zakres najlepszych wartości hiperparametrów, a następnie dokładnie przeszukano ten zakres za pomocą algorytmu Optuna [3]. W przypadku modeli implicit ALS strojenie obejmowało hiperparametry α, β i γ odpowiedzialne za wagę pewności, liczbę wymiarów latentnych f oraz siłę regularizacji λ. Liczba iteracji algorytmu ALS została ustalona na 30, ze względu na brak istotnej poprawy wyników przy większej liczbie iteracji podczas wstępnego przeszukiwania siatkowego. Natomiast dla modelu BPR strojenie obejmowało f, λ, liczbę iteracji oraz długość kroku. W przypadku modelu popularnościowego nie przeprowadzano strojenia ze względu na brak hiperparametrów. Dla każdego z optymalizowanych modeli przeprowadzono 150 prób algorytmu Optuna w jednej serii. Aby wykluczyć ryzyko znalezienia suboptymalnych rozwiązań, po każdej serii oceniano stabilność metryki celowej oraz stopień skupienia uzyskanych hiperparametrów wokół znalezionego optimum. W sytuacjach, gdy najlepsze próby wykazywały dużą zmienność wartości hiperparametrów lub gdy znaleziona kombinacja znajdowała się na granicy przeszukiwanego przedziału, procedurę powtarzano z rozszerzonym lub przesuniętym zakresem aż do uzyskania spójnych wyników. Każdy etap przeprowadzono z ustalonym ziarnem generatora liczb pseudolosowych, co gwarantuje pełną reprodukowalność procesu optymalizacji.

5.4. Finalna ewaluacja testowa

Po wybraniu optymalnych wartości hiperparametrów dla każdego z modeli policzono odchylenie standardowe metryk dla 5 podziałów na zbiór treningowy i walidacyjny w celu sprawdzenia stabilności modeli ze względu na losowość podziału danych. Następnie zostały one ponownie wytrenowane na połączonym zbiorze treningowym i walidacyjnym. Po czym poddano je ewaluacji na podstawie metryk precyzji (precision), odtworzenia (recall) i NDCG, będącymi metrykami trafności rekomendacji, a także metryki pokrycia (coverage), różnorodności (diversity) i różnicy w popularności rekomendacji (ΔGAP). Każda z metryk została obliczona dla listy rekomendacji o długości 10, co jest standardem w dziedzinie [9, 19], a następnie uśredniona po wszystkich użytkownikach. Ewaluację powtórzono 5 razy z różnym ziarnem losowym modelu, a wyniki uśredniono oraz obliczono odchylenie standardowe. Dodatkowo metryka NDCG@10 została wyznaczona dla indywidualnie dla każdego użytkownika, co pozwoliło na przeprowadzenie testu Wilcoxona dla par obserwacji w celu sprawdzenia istotności statystycznej różnic jakości rekomendacji między modelami.

5.5. Analiza wrażliwości na α, β i γ

Ze względu na szczególnie zainteresowanie wpływem wag pewności na jakość rekomendacji, przeprowadzono dodatkową analizę parametru odpowiedzialnego za ich skalowanie, czyli α. W tym celu dla każdej funkcji pewności w modelach implicit ALS przeprowadzono eksperymenty, w których obliczono NDCG@10 dla 25 wartości tego hiperparametru w równych odstępach na siatce logarytmicznej wokół optymalnej wartości znalezionej w procesie strojenia. Analogiczny eksperyment przeprowadzono dla 40 wartości parametru β w funkcji $c_{ui} = 1 + \alpha \log(1 + r_{ui} + \beta)$, w celu sprawdzenia wrażliwości modelu na jego zmianę. Utworzono również dodatkowe wykresy przedstawiające zależność funkcji pewności od zmiennej `playtime_forever` oraz wyznaczono jej efektywny zakres dla poszczególnych modeli Implicit ALS. Pozwoliło to na lepsze zrozumienie, w jaki sposób zastosowane przekształcenia wpływają na proces uczenia modeli.

Ze względu na szczególne zainteresowanie wpływem wag pewności na jakość rekomendacji, przeprowadzono dodatkową analizę parametrów odpowiedzialnych za ich skalowanie i przesunięcie, czyli α, β oraz γ. W tym celu dla każdej funkcji pewności w modelach implicit ALS przeprowadzono eksperymenty, w których obliczono NDCG@10 dla 25 wartości hiperparametru α w równych odstępach na siatce logarytmicznej wokół optymalnej wartości znalezionej w procesie strojenia. Analogiczne badanie wrażliwości przeprowadzono dla 40 wartości parametru β w funkcji $c_{ui} = 1 + \alpha \log(1 + r_{ui} + \beta)$ oraz 40 wartości parametru γ w funkcji $c_{ui} = 1 + \gamma$. W rozdziale 6.2 utworzono również dodatkowe wykresy przedstawiające zależność funkcji pewności od zmiennej `playtime_forever` oraz wyznaczono jej efektywny zakres dla poszczególnych modeli implicit ALS. Pozwoliło to na lepsze zrozumienie, w jaki sposób zastosowane przekształcenia wpływają na proces uczenia modeli.

5.6. Ograniczenia metodologii

Prezentowane badanie posiada pewne ograniczenia, które mogą wpływać na interpretację wyników. Przede wszystkim eksperyment został przeprowadzony na jednym zbiorze

danych, by uzyskać w pełni reprezentatywne wyniki, należałoby powtórzyć go na innych zbiorach danych. W zbiorze nie pojawiła się informacja o dacie przypisania poszczególnych gier do bibliotek użytkowników, co uniemożliwiło przeprowadzenie podziału danych w sposób chronologiczny oraz uwzględnienia zmiany preferencji użytkowników w czasie. Ponadto ewaluacja została przeprowadzona na wszystkich graczach bez podziału na grupy ze względu na liczbę interakcji, co mogłoby ujawnić różnice w jakości rekomendacji dla użytkowników o różnym poziomie aktywności. Dodatkowo ewaluacja została przeprowadzona w sposób offline, a nie w sposób testu A/B. Na koniec warto uwzględnić, że ze względu na czasochłonne obliczenia oraz ilość trenowanych modeli podczas analizy wrażliwości na α i β pozostałe hiperparametry pozostały na stałym poziomie, który wybrano na podstawie modelu z największym NDCG@10 w wybranej rodzinie.

6. Wyniki eksperymentów

6.1. Wyniki strojenia hiperparametrów

Wyniki strojenia hiperparametrów dla każdego z modeli implicit ALS zostały przedstawione w tabeli 6.1. Tak jak wspomniano w metodologii parametr odpowiadający za ilość iteracji jest równy 30 w każdym z modeli.

Tabela 6.1: Najlepsze hiperparametry wariantów ALS wybrane na zbiorze walidacyjnym

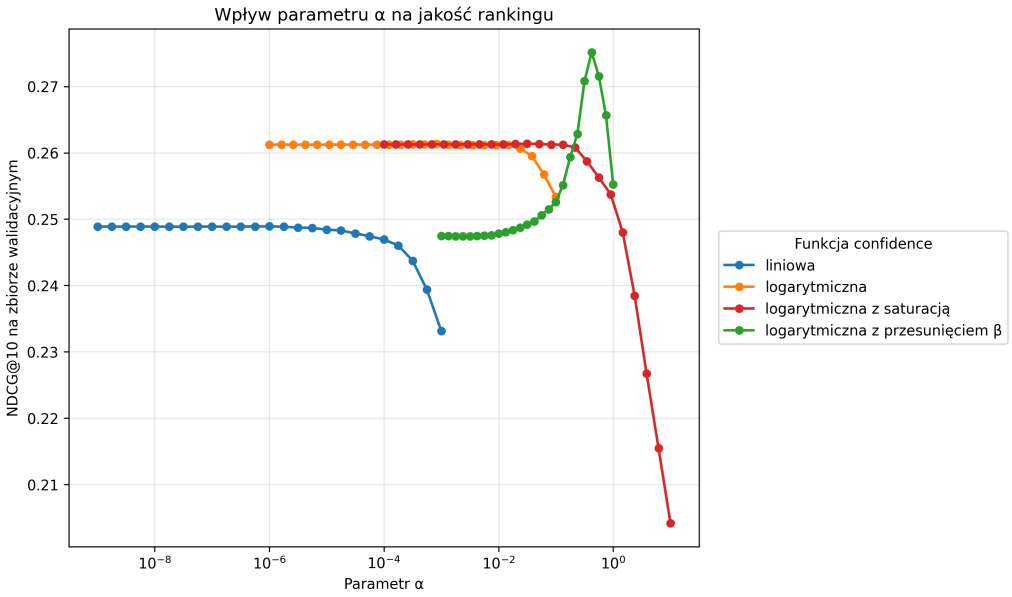
Wariant	Czynniki	α	β	Regularyzacja	NDCG@10
Liniowa	80	$9.948 \cdot 10^{-7}$	-	0.003588	0.249
Logarytmiczna	24	0.000571	-	0.130920	0.261
Logarytmiczna z saturacją	24	0.024390	-	0.169714	0.261
Logarytmiczna z przesunięciem β	48	0.389185	108.284	0.010565	0.275

Źródło: opracowanie własne na podstawie wyników Optuny.

Warto zwrócić szczególną uwagę na niskie wartości parametru α odpowiadającego za skalowanie wagi pozytywnych interakcji. Jako standardową wartość tego hiperparametru zwykle przyjmuje się 1 [4, 8], a przeszukanie przeprowadza wsród większych wartości w celu zróżnicowania dlugich interakcji od ich braku. Konsekwencją tego jest bardzo małe wykorzystanie potencjalnych informacji o preferencjach zawartych w długości czasu gry. Taki wynik był spodziewany w przypadku liniowego przekształcenia ze względu na przypisywanie ogromnych wag pewności dla obserwacji z długiego ogona. Klasycznym rozwiązaniem tego problemu jest zastosowanie logarytmu, jednak pomimo poprawy NDCG@10 α nadal pozostała mała. Z tego powodu zdecydowano się na zastosowanie przekształcenia $c_{ui} = 1 + \alpha \frac{\log(1+r_{ui})}{1+\log(1+r_{ui})}$ ograniczającego wartości zmiennej przed skalowaniem do przedziału $[0, 1)$. Nie miało to jednak znaczącego wpływu na metrykę NDCG@10 na zbiorze walidacyjnym, a dodatkowe eskperymenty pokazały marginalność różnic między tymi modelami. Wyniki poprzednich eksperymentów doprowadziły do sformułowania hipotezy, że niskie wartości `playtime_forever` wprowadzały znaczny szum do modeli stosujących logarytmiczne przekształcenia, co w efekcie zmuszało dobór niewielkich α w procesie optymalizacji hiperparametrów. W celu zniwelowania tego zjawiska oraz nadania interakcjom z `playtime_forever = 0` zastosowano $c_{ui} = 1 + \alpha \log(1 + r_{ui} + \beta)$. W procesie optymalizacji β została ustalona na wartość 108,284, co umożliwiło skuteczne różnicowanie siły interakcji, bez ryzyka zdominowania procesu uczenia przez szum informacyjny krótkich sesji bądź przez dużą skalę wartości długich interakcji.

W celu lepszego zobrazowania wpływu tych dwóch hiperparametrów na modele wygenerowano wykres 6.1 oraz 6.2 przedstawiające NDCG@10 od ich wartości wokół znalezionego

optimum.

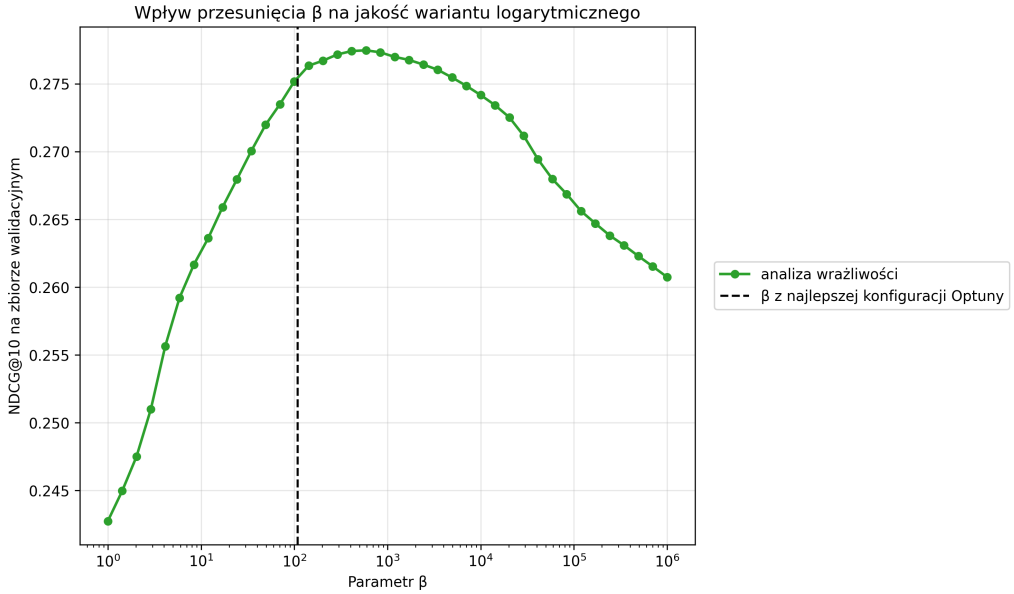


Rysunek 6.1: Wrażliwość modeli na hiperparametr α

Na wykresie 6.1 widać, że dla każdej funkcji pewności z wyjątkiem logarytmicznej z przesunięciem β wartości NDCG@10 są najwyższe dla $\alpha \to 0$, co sugeruje zredukowanie potencjalnego sygnału o preferencjach użytkowników zawartego w zmiennej `playtime_forever` do sygnału binarnego odpowiadającego posiadaniu i nieposiadaniu danej gry. W celu zweryfikowania tej hipotezy w kolejnym podrozdziale zostały przebadane funkcje wag pewności zastosowane w modelach o optymalnych hiperparametrach. Jedynie dla wariantu implicit ALS z dodatkowym hiperparametrem znalezione optimum α nie leży w otoczeniu 0. W związku z tym jako jedyny jest w stanie wyciągnąć dodatkową informację o preferencjach z długości interakcji użytkowników z grami. Należy jednak podkreślić, że przyrost NDCG@10 w stosunku do zdegenerowanych modeli jest umiarkowany, co sugeruje, że posiadanie danego tytułu w bibliotece niesie dostateczny sygnał o preferencjach do generowania rekomendacji satysfakcjonującej jakości.

Analogiczną zależność od parametru β przedstawiono na wykresie 6.2. Widać na nim, że wraz ze wzrostem hiperparametru NDCG@10 rośnie, najprawdopodobniej ze względu na efekt tłumienia szumu w danych, następnie stabilizuje się dla wartości rzędu od 10^2 do 10^3. Taka wartość stanowi kompromis między efektem tłumienia, a możliwością wykorzystania wysokich wartości zmiennej `playtime_forever` w procesie dopasowania modelu. Dalsze zwiększanie wartości hiperparametru pogarsza jakość rekomendacyjną modelu, ponieważ podobnie jak w przypadku małych wartości α prowadzą one do niewykorzystnia informacji o preferencji płynącej z długości interakcji. Warto podkreślić, że β zwiększa wartość c_{ui} obserwacji z `playtime_forever` $= 0$, która w pozostałych modelach zawsze wynosi 1. Na wykresie można również zauważyć, że hiperparametry wybrane przez algorytm Optuny nie są w pełni optymalne, jendak ze względu na marginalność różnic eksperyment był kontynuowany bez ich zmiany.

Wartości hiperparametrów otrzymane w procesie strojenia modelu BPR zostały przedstawione w tabeli 6.2.



Rysunek 6.2: Wrażliwość modelu na hiperparametr β

Tabela 6.2: Najlepsze hiperparametry modelu BPR wybrane na zbiorze walidacyjnym

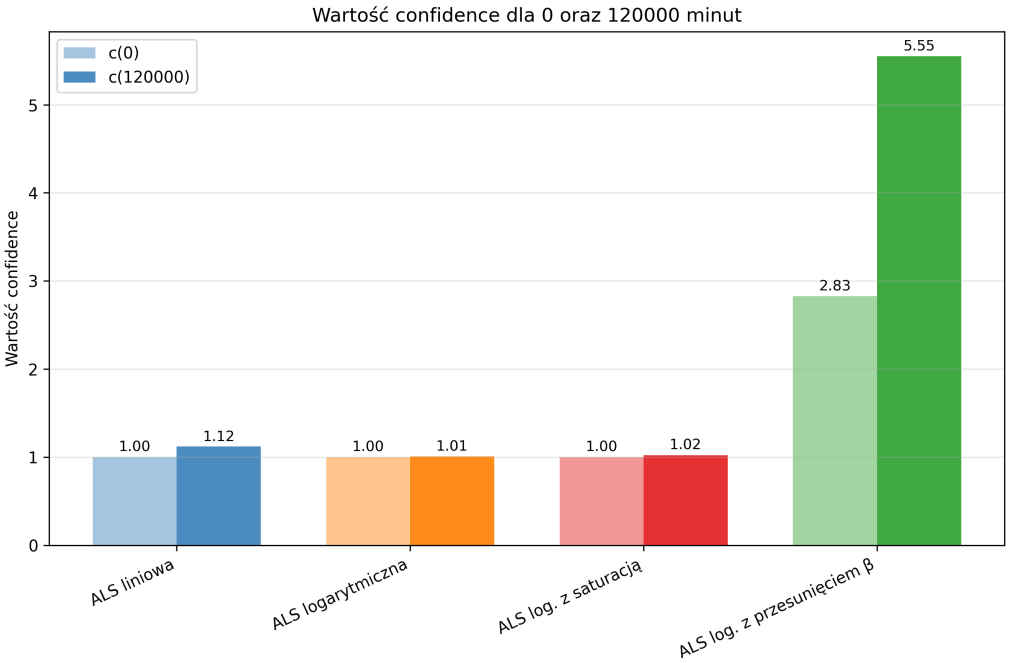
Wariant	Czynniki	Krok uczenia	Regularyzacja	Iteracje	NDCG@10
BPR	96	0.001760	0.003237	150	0.212

Źródło: opracowanie własne na podstawie wyników Optuny.

W przypadku modelu BPR nie jest możliwa tak dogłębna analiza dobranych hiperparametrów, ze względu na to, że interakcje traktowane są w sposób binarny. Regularyzacja i długość kroku uczenia wpływają wyłącznie na proces znajdywania maksimum prawdopodobieństwa aposteriori cech ukrytych. Jedynie wymiar przestszeni cech ukrytych pozwala na interpretacje złożoności relacji między użytkownikami, a produktami. Już na tym etapie można zauważyć znaczące różnice między metryką NDCG@10 między klasami modeli, jednak ich analizę przeprowadzono w późniejszych rozdziałach.

6.2. Analiza funkcji wag pewności

Aby zweryfikować hipotezę o redukcji sygnału z `playtime_forever` w przypadku modeli implicit ALS przeprowadzono dodatkową analizę wag pewności otrzymanych w modelach. Na wykresie 6.3 przedstawiono ich zakres wartości dla każdego z modeli.

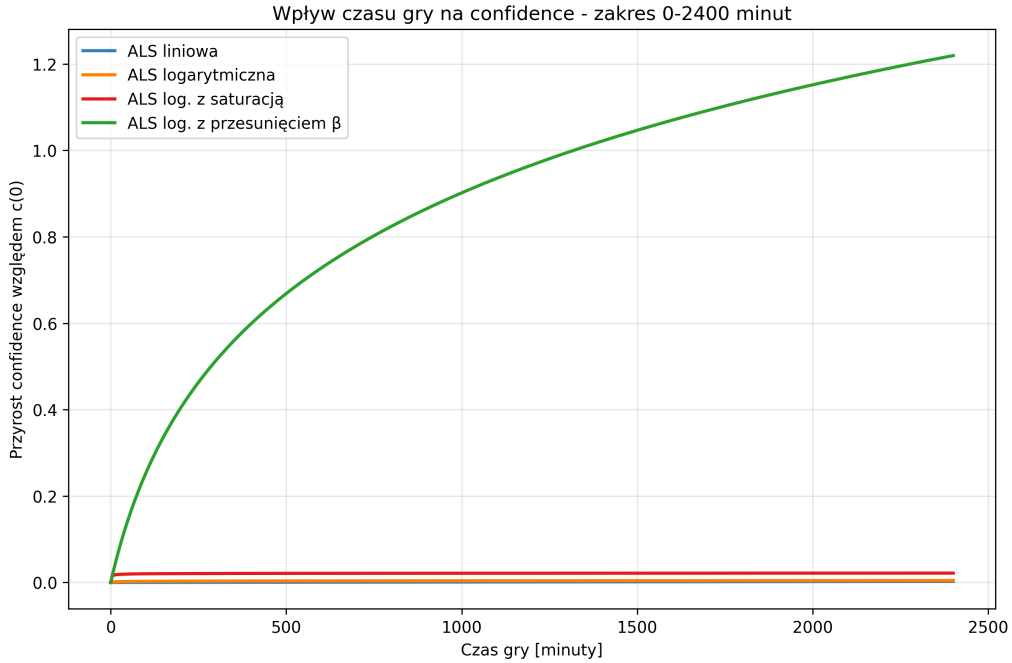


Rysunek 6.3: Zakres przyjmowanych wartości wag pewności

W przypadku modelu z logarytmem oraz logarytmem z saturacją wykres 6.3 potwierdza, że długości interakcji mają bardzo znikomy wpływ na proces uczenia. Podobnie w przypadku modelu z liniową funkcją pewności zakresy wartości są niewielkie. Jedynie model z logarytmicznym przekształceniem z przesunięciem β po wytrenowaniu uzyskał zakres pewności, który najdłuższym interakcjom przypisuje dwukrotnie większą wartość niż tym bez zarejestrowanego czasu gry. Umożliwia to zróżnicowanie siły sygnału w zależności od zachowania użytkownika. Należy jednak podkreślić, że w procesie optymalizacji modelu Implicit ALS uwzględniane są wszystkie możliwe kombinacje par użytkownik-gra. Ze względu na niską gęstość macierzy interakcji wynoszącą 0,00887, przypadki niezaobserwowane, dla których $c_{ui} = 1$ dominują w procesie uczenia. W takim przypadku zmiana wskaźnika c_{ui} do poziomu 2,8-5,5 dla nielicznych interakcji pozytywnych ma nieznaczny wpływ na globalną wartość funkcji straty. Sugeruje to, że przewaga wariantu z przesunięciem β wynika w dużej mierze ze zdolności do jednoczesnej kompresji zmienności sygnału dla niskich war-

tości czasu rozgrywki oraz ograniczenia wpływu wartości skrajnych, które w pozostałych modelach jedynie pogarszają jakość rekomendacji w przypadku uwzględnienia zmienności `playtime_forever` w procesie uczenia.

Aby przedstawić tą własność stworzono wykres 6.4 przedstawiający przyrost wartości wag pewności w modelach implicit ALS w zależności od zmiennej `playtime_forever` dla pierwszych 20 godzin rozgrywki.



Rysunek 6.4: Wartości wag pewności w zależności od zmiennej `playtime_forever`

Przebieg krzywych na wykresie 6.4 obrazuje opisywaną stabilność wariantu z przesunięciem β. Podczas gdy dla pozostałych modeli funkcja przyrostu wartości zaufania pozostaje bliska 0, model z $c_{ui} = 1 + \alpha \ln(r_{ui} + 1 + \beta)$ wykazuje wyraźny wzrost. W przedziale pierwszych kilkuset minut rozgrywki przyrost wartości c_{ui} jest stosunkowo szybki, natomiast w dalszej części wykresu krzywa widocznie łagodnieje. Te dwie właśności pozwala wyciągnąć możliwie najwięcej sygnału o preferencjach z pierwszych godzin spędzonych w danym tytule, bez nadawania nieproporcjonalnych wag zarówno dla małych jak i dużych wartości `playtime_forever`.

Warto rozważyć teoretyczny przyrost pozostałych wariantów transformacji w przypadku, gdyby nie uległy one skrajnemu spłaszczeniu. Dla modelu z $c_{ui} = 1 + \alpha \log(r_{ui} + 1)$ dla standardowego hiperparametru $\alpha = 1$ zaledwie 19 minut rozgrywki wystarczy, aby wartość pewności modelu gwałtownie skoczyła od poziomu bazowego równego 1 aż do 4. Równocześnie, aby uzyskać wzrost z poziomu 7 do 8, użytkownik musi spędzić w grze w przybliżeniu dodatkowe 11 i pół godziny. Oznacza to, że ta funkcja wykazuje się znaczną czułością w wąskim przedziale małych wartości, podczas gdy zmiana wsród dużych wartości jest nieznaczna. Podobny efekt ma $c_{ui} = 1 + \alpha \frac{\log(1+r_{ui})}{1+\log(1+r_{ui})}$. Natomiast w przypadku liniowej funkcji pewności $c_{ui} = 1 + \alpha r_{ui}$ to zjawisko przybiera odwrotny charakter. Ponieważ jej przyrost jest proporcjonalny do spędzonego czasu, model nie posiada efektu saturacji. W efekcie użytkownicy generujący skrajnie wysokie liczby godzin mają dominujący wpływ na wartość funkcji straty. Model w efekcie zamiast różnicować zachowania, w

procesie optymalizacji ulega binaryzacji pod wpływem wartości odstających.

6.3. Jakość rekomendacji na zbiorze testowym

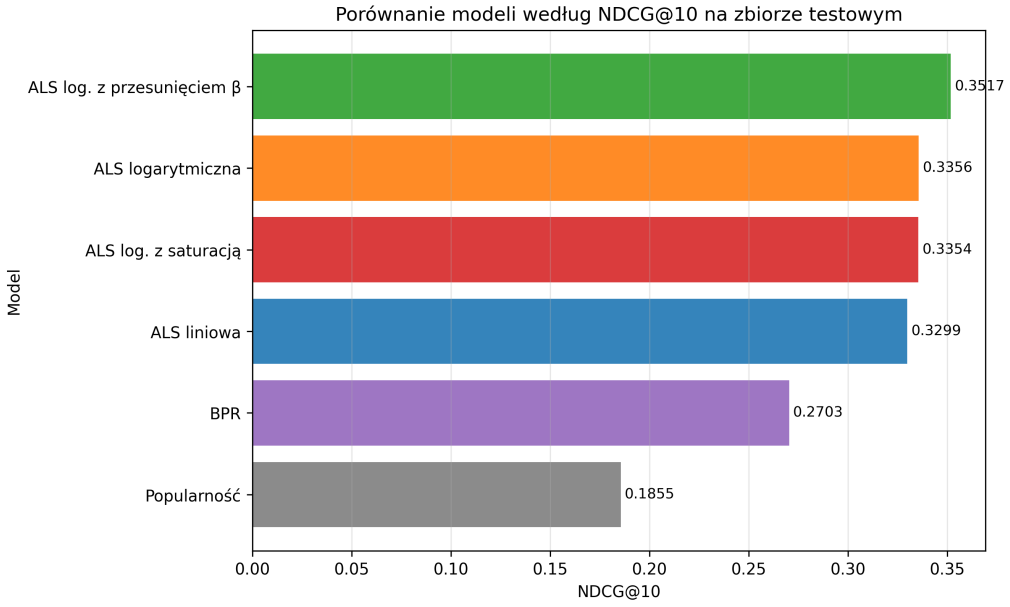
Przedstawione różnice w procesie uczenia mają swoje odzwierciedlenie w metrykach otrzymanych na zbiorze testowym. W tabeli 6.3 przedstawiono uśrednione wyniki otrzymane dla każdego z modeli. Odchylenia standardowe metryk zostaną opisane w późniejszym podrozdzialne, gdyż zostały usunięte z tabeli na rzecz czytelności.

Tabela 6.3: Wyniki modeli na zbiorze testowym

Model	NDCG@10	Precision@10	Recall@10	Coverage@10	ΔGAP@10	Diversity@10
Popularność	0.1855	0.0965	0.1783	0.0087	1.4233	-
ALS liniowa	0.3299	0.1841	0.2642	0.1178	-0.2112	0.7854
ALS logarytmiczna	0.3356	0.1810	0.3019	0.0762	0.5433	0.5963
ALS log. z saturacją	0.3354	0.1809	0.3019	0.0761	0.5443	0.5948
ALS log. z przesunięciem β	0.3517	0.1956	0.3338	0.1313	0.4191	0.7314
BPR	0.2703	0.1329	0.2614	0.3039	-0.0789	0.5499

Źródło: opracowanie własne. Wartości dla ALS i BPR to średnie z pięciu uruchomień.

Ze względu na dużą ilość metryk w tym podrozdziale skupiono się na metrykach rankingowych, pozostałe z nich zostaną przeanalizowane w kolejnym podrozdziale poświęconym kompromisom między nimi. Ze względu na stosowanie metryki NDCG@10 jako głównej metryki rankingowej w celu lepszego zaprezentowania różnic między modelami utworzono wykres 6.5, który przedstawia jej wartość dla każdego z testowanych modeli.



Rysunek 6.5: Wartości NDCG@10 każdego z modeli na zbiorze testowym

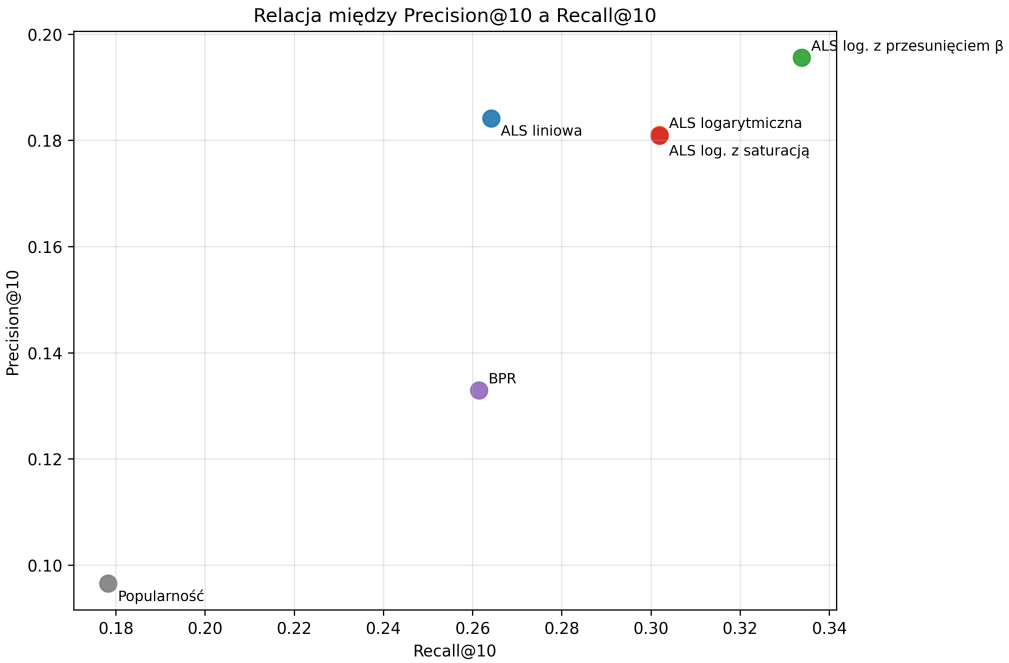
Wykres jednoznacznie pokazuje, że model popularnościowy cechuje się niższą zdolnością rekomendacji niż modele oparte na metodach faktoryzacji macierzy. Mimo to uzyskany wynik nie jest zaniedbywalny. Model popularnościowy, efektywnie wykorzystuje silny efekt

nierównomierności popularności gier w analizowanym zbiorze danych przedstawiony na wykresie 4.3. Ze względu na to został on wykorzystany jako punkt odniesienia w niniejszej pracy.

Uzyskana przez BPR wartość NDCG@10 jest najniższa spośród wszystkich modeli faktoryzacji. Należy jednak podkreślić, że nie oznacza to, że model jest obiektywnie gorszy. W ocenie systemów rekomendacyjnych należy również uwzględniać metryki odpowiedzialne za pokrycie katalogu i podatność na bias popularności.

Istotnych wniosków dostarcza porównanie poszczególnych wariantów modelu ALS między sobą. Najwyższą wartość NDCG@10 uzyskał model z przesunięciem β (0,3517), co stanowi poprawę o 4,8% względem wariantu logarytmicznego (0,3356) oraz o 6,6% względem wariantu liniowego (0,3299). Uzyskana poprawa jakości rankingu względem bazowej implementacji ALS wskazuje, że zastosowana modyfikacja ma istotny wpływ na działanie modelu, mimo zachowania tej samej architektury. Przeprowadzono również testy Wilcoxona dla par obserwacji z korektą Holma-Bonferroniego, aby potwierdzić statystyczną istotność różnic. Otrzymano w nich $p < 0.001$, zatem różnice są istotne. Warto zauważyć, że podobne obserwacje zostały przedstawione w pracy [20], gdzie autorzy pokazali, że odpowiednio dostrojone i zmodyfikowane warianty klasycznego implicit ALS mogą osiągać wyższe wyniki, a nawet konkurować z bardziej złożonymi modelami rekomendacyjnymi, w tym neuronowymi.

Rola metryk precyzji i odtworzenia jest ograniczona ze względu na ich znaczne skorelowanie z NDCG@10 wynikające z faktu, że wszystkie te metryki oceniają trafność rekomendacji na podstawie ilości trafień elementów istotnych dla użytkownika. Jednak mogą one pokazać różnice w jakości rekomendacji w zależności od poziomu aktywności użytkownika.



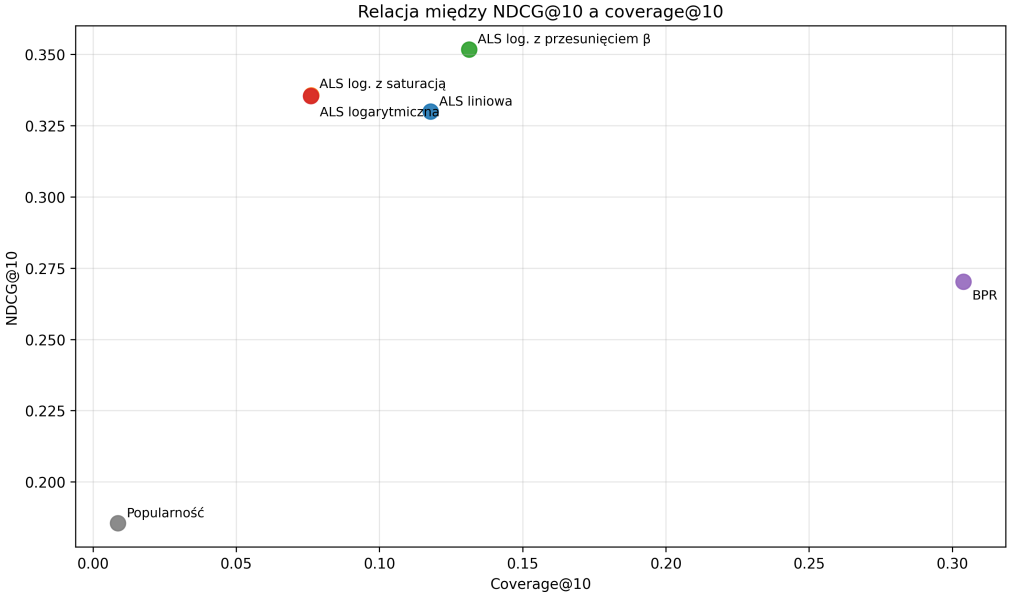
Rysunek 6.6: Wartości precyzji względem odtworzenia na zbiorze testowym

Z wykresu 6.6 można zauważyć, że pomimo niższej wartości metryki odtworzenia od modeli z przekształceniem logarytmicznym i logarytmicznym z saturacją model liniowy ma wyższą wartość precyzji. Może to sugerować, że ten model oferuje trafniejsze reko-

mendacje w przypadku użytkowników z większą liczbą interakcji. Weryfikacja tej hipotezy wymagałaby podziału użytkowników według liczby posiadanych interakcji, co wykracza poza zakres niniejszej pracy. Widoczne jest również, że model implicit ALS z dodatkowym parametrem β wypada najlepiej w kontekście obu analizowanych na wykresie metryk.

6.4. Analiza kompromisów jakości rekomendacji

W procesie ewaluacji systemów rekomendacyjnych istotne jest uwzględnienie również metryk wykraczających poza trafność predykcji (ang. beyond-accuracy), takich jak pokrycie katalogu, różnorodność oraz ΔGAP. W niniejszym rozdziale poddano analizie kompromisy między jakością rekomendacji a wspomnianymi wyżej miarami.

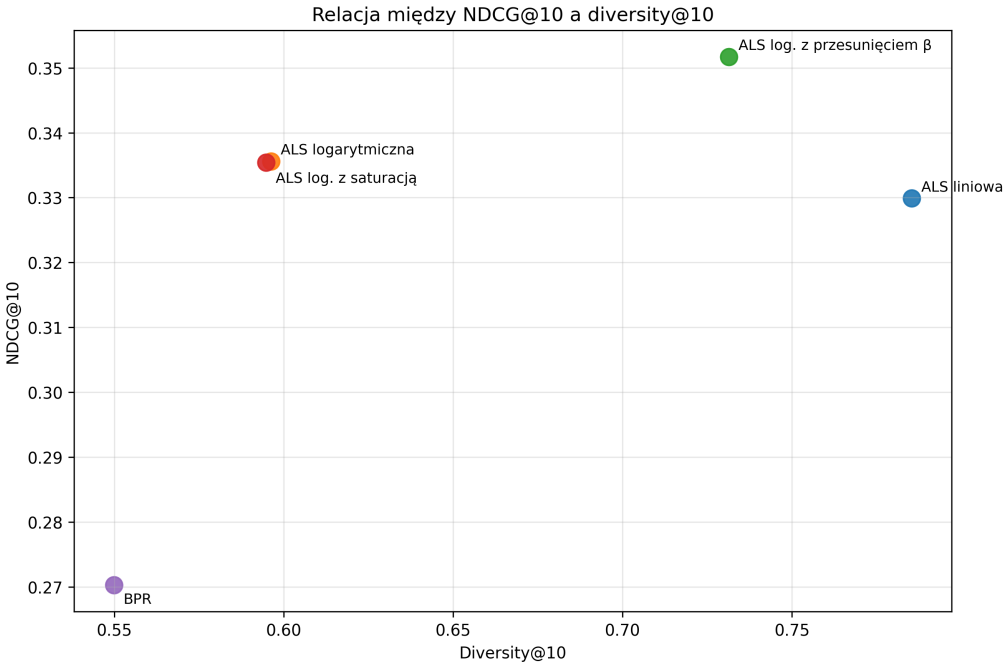


Rysunek 6.7: Wartości pokrycia katalogu względem NDCG@10 na zbiorze testowym

W przypadku metryki pokrycia na wykresie 6.7 widoczna jest znacząca rozbieżność między każdym z badanych typów modeli. Model BPR wykazuje się najwyższym pokryciem (0,3039), będącym ponad dwukrotnie większym niż każdy wariant ALS, jednak osiąga to kosztem niższej wartości NDCG@10. Rodzina modeli implicit ALS tworzy wyraźne skupienie na wykresie, wewnątrz którego można zaobserwować poprawę wraz z zastosowaniem przesunięcia β. Wariant ten wypada najlepiej spośród testowanych wariantów ALS pod względem obu przedstawianych metryk. Osiąga nie tylko najwyższą wartość NDCG@10, ale również wyższe pokrycie katalogu (0,1313) niż warianty logarytmiczne (0,0762) oraz wariant liniowy (0,1178). Niski wynik wariantu popularnościowego pod względem obu metryk jest spójny z oczekiwaniami i wynika z jego niepersonalizowanego charakteru.

Po analizie pokrycia katalogu warto przeanalizować metrykę różnorodności, opisującą zróżnicowanie pozycji rekomendowanych pojedynczemu użytkownikowi, mierzone na podstawie odległości między wektorami ukrytymi rekomendowanych przedmiotów. Model popularnościowy został pominięty w tym porównianiu ze względu na brak generowanych przez niego wektorów latentnych.

Na wykresie 6.8 warto zauważyć, że model z przesunięciem β osiąga najwyższą war-

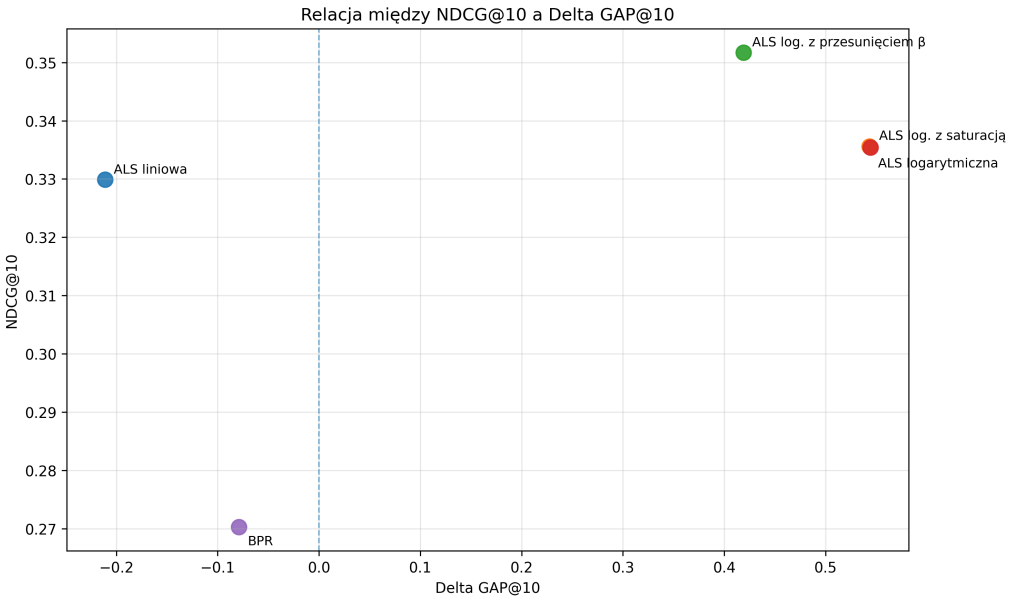


Rysunek 6.8: Średnie wartości różnorodności względem NDCG@10 na zbiorze testowym

tość NDCG@10, jak i wysoką wartość różnorodności, mimo wykorzystania przestrzeni o mniejszej wymiarowości (48) niż wariant liniowy (80) i model BPR (96). Zestawienie liczby czynników ukrytych z wartością różnorodności nie wskazuje na monotoniczną zależność między wymiarowością przestrzeni latentnej a zróżnicowaniem rekomendacji. W przypadku modelu BPR, mimo najwyższej liczby czynników, uzyskano najniższą wartość różnorodności wśród porównywanych modeli, podczas gdy wariant liniowy, dysponujący mniejszą przestrzenią ukrytą (80 czynników), osiągnął najwyższy wynik. Sugeruje to, że o zróżnicowaniu rekomendacji decyduje w większym stopniu charakter funkcji celu optymalizacji niż sama liczba wymiarów. Warianty logarytmiczne, mając najmniejszą liczbę czynników (24) uzyskały również najniższą wartość Diversity spośród modeli implicit ALS. W tym przypadku nie można jednoznacznie stwierdzić, czy niższe zróżnicowanie wynika ze zmniejszonej wymiarowości przestrzeni latentnej, czy z wpływu przekształcenia. Należy podkreślić, że liczba czynników dla każdego wariantu została dobrana niezależnie w procesie strojenia hiperparametrów, w związku z czym zaobserwowane różnice w wartości Diversity mogą częściowo wynikać z odmiennej wymiarowości przestrzeni ukrytej, a nie wyłącznie z charakteru zastosowanej funkcji wagowej. Rozdzielenie obu efektów wymagałoby przeprowadzenia dodatkowego eksperymentu ze stałą liczbą czynników dla wszystkich porównywanych wariantów. Dodatkowo kontrast między metryką pokrycia zbioru a różnorodnością w modelu BPR może wskazywać na to, że rekomendacje generowane przez ten model różnią się znacząco między poszczególnymi użytkownikami, natomiast w obrębie rekomendacji dla pojedyńczego użytkownika pozostają stosunkowo podobne do siebie.

Na koniec pozostała do przeanalizowania zależność NDCG@10 od ΔGAP, czyli miary obciążenia rekomendacji popularnością. Jej optymalna wartość wynosi 0 i odpowiada sytuacji, w której popularność rekomendowanych gier jest zbliżona do popularności gier w bibliotece użytkownika zawartych w zbiorze treningowym. Na wykresie 6.9, model popularnościowy został ponownie pominięty, aby zachować czytelność pozostałych wyników.

Wartość ΔGAP dla tego modelu wynosi 1,4233, co stanowi maksymalne możliwe obciążenie rekomendacji popularnością w badanym zbiorze.



Rysunek 6.9: Średnie wartości ΔGAP względem NDCG@10 na zbiorze testowym

Modele BPR oraz implicit ALS z liniową funkcją pewności uzyskały ujemną wartość ΔGAP co jest nietypowym zachowaniem dla systemów rekomendacyjnych, które mają tendencję do faworyzowania popularnych produktów. Może to być efektem większej wymiarowości przestrzeni wektorów ukrytych obu modeli, która pozwala na dokładniejsze reprezentowanie niszowych cech i preferecji. Również mniejsza wartość hiperparametru λ odpowiadającego za regularyzację na tle pozostałych modeli pozwala na znaczny wzrost norm euklidesowych wektorów reprezentacji cech gier i preferencji użytkowników. Umożliwia to tworzenie modelowi ekstremalnych dopasowań dla niszowych gier o niskiej popularności. W przypadku pozostałych modeli ALS ich metrki ΔGAP są większe od zera, więc rekomendują one przedmioty popularniejsze niż w zbiorze treningowym użytkownika, ale za to cechują się większą trafnością rekomendacji. Na wykresie 6.9 widać również, że dodany hiperparamet β nie tylko poprawił trafność, ale również zmniejszył obciążenie popularnością.

6.5. Stabilność wyników

W celu sprawdzenia stabilności wyników obliczono odchylenia standardowe na pięciu uruchomieniach algorytmów implicit ALS i BPR. Wyniki przedstawiono w tabelach 6.4.

Analiza odchyleń standardowych uzyskanych w pięciu losowych inicjalizacjach algorytmow implicit ALS i BPR potwierdza wysoką stabilność wszystkich badanych modeli. W przypadku metryk trafności rekomendacji wartości odchylenia standardowego nie przekraczają $0,05\%$. Świadczy to o tym, że przedstawione różnice jakościowe między modelami są w pełni nieprzypadkowe. Z kolei najwyższe odchylenie względne zaobserwowano przy metryce ΔGAP@10 dla modelu BPR, które wynosi $6,84\%$ przy wartości średniej równej $-0,0789$. Ze względu na znikome różnice wynikające z odchyleń wykresy mające na celu

Tabela 6.4: Wyniki ewaluacji modeli na zbiorze testowym w podziale na metryki dokładności oraz metryki profilowe i różnorodności

Wariant	Precision@10	Recall@10	NDCG@10
Liniowa	0.1841 ± 0.0003	0.2642 ± 0.0005	0.3299 ± 0.0005
Logarytmiczna	0.1810 ± 0.0001	0.3019 ± 0.0002	0.3356 ± 0.0003
Logarytmiczna z saturacją	0.1809 ± 0.0002	0.3019 ± 0.0003	0.3354 ± 0.0005
Logarytmiczna z przesunięciem β	0.1956 ± 0.0002	0.3338 ± 0.0003	0.3517 ± 0.0004
BPR	0.1329 ± 0.0003	0.2614 ± 0.0005	0.2703 ± 0.0006
Wariant	**Coverage@10**	**Delta GAP@10**	**Diversity@10**
Liniowa	0.1178 ± 0.0007	-0.2112 ± 0.0016	0.7854 ± 0.0063
Logarytmiczna	0.0762 ± 0.0003	0.5433 ± 0.0008	0.5963 ± 0.0159
Logarytmiczna z saturacją	0.0761 ± 0.0002	0.5443 ± 0.0009	0.5948 ± 0.0152
Logarytmiczna z przesunięciem β	0.1313 ± 0.0009	0.4191 ± 0.0119	0.7314 ± 0.0174
BPR	0.3039 ± 0.0012	-0.0789 ± 0.0054	0.5499 ± 0.0009

Źródło: opracowanie własne.

je przedstawić zostały pominięte.

Tabela 6.5: Wyniki ewaluacji modeli na zbiorze walidacyjnym w podziale na metryki dokładności oraz metryki profilowe i różnorodności

Wariant	Precision@10	Recall@10	NDCG@10
Liniowa	0.1360 ± 0.0006	0.2302 ± 0.0008	0.2466 ± 0.0011
Logarytmiczna	0.1349 ± 0.0009	0.2589 ± 0.0019	0.2534 ± 0.0018
Logarytmiczna z saturacją	0.1350 ± 0.0008	0.2596 ± 0.0015	0.2536 ± 0.0015
Logarytmiczna z przesunięciem β	0.1479 ± 0.0005	0.2963 ± 0.0012	0.2744 ± 0.0011
Wariant	**Coverage@10**	**Delta GAP@10**	**Diversity@10**
Liniowa	0.1025 ± 0.0003	-0.2048 ± 0.0010	0.7792 ± 0.0064
Logarytmiczna	0.0636 ± 0.0006	0.4899 ± 0.0071	0.6048 ± 0.0044
Logarytmiczna z saturacją	0.0635 ± 0.0006	0.4942 ± 0.0047	0.6045 ± 0.0040
Logarytmiczna z przesunięciem β	0.1138 ± 0.0017	0.4072 ± 0.0119	0.7549 ± 0.0146

Źródło: opracowanie własne.

W związku z zastosowaniem wydzielonego zbioru testowego zamiast procedury walidacji krzyżowej, przeprowadzono analizę stabilności względem losowego podziału danych. W tabeli 6.5 przedstawione są wartości średnie metryk i odchylenia standardowe liczone na pięciu losowych podziałach na zbiory treningowe i walidacyjne.Największą względną wartość odchylenia standardowego odnotowano dla metryki Delta GAP@10 w wariancie logarytmicznym z przesunięciem β, która wynosi $2,92\%$. Wynika to najprawdopodobniej ze znacznego rozmiaru wykorzystanego zbioru danych, który uodparnia proces uczenia modeli na różnice wynikające z podziału losowego. Ze względu na niskie wartości odchyleń

wykresy mające na celu je zobrazować ponownie zostały pominięte.

7. Wnioski

7.1. Podsumowanie eksperymentów

Zakres eksperymentu objął dostrajanie hiperparametrów czterech wariantów modeli implicit ALS z różnymi wagami pewności (liniowa, logarytmiczna, logarytmiczna z saturacją, logarytmiczna z przesunięciem β), modelu BPR oraz popularnościowego jako odniesienie. Dobór hiperparametrów pokazał, że w przypadku modeli iALS optymalny hiperparametr $\alpha \to 0$, co w praktyce redukowało sygnał o preferencji płynący z `playtime_forever` do wariantu binarnego (użytkownik posiada/nie posiada gry w bibliotece). Jedynym wyjątkiem był model z $c_{ui} = 1 + \alpha \log(1 + r_{ui} + \beta)$, który w umiarkowanym stopniu różnicował sygnał wynikający z czasu gry. Było to możliwe ze względu na dodatkowy hiperparametr β, który pozwalał tłumić duże zmiany wartości c_{ui} wśród niskich wartości czasu rozgrywki oraz jednocześnie ograniczyć wpływ dużych wartości odstających stosując przekształcenie logarytmiczne. Model z przesunięciem osiągnął najwyższe wyniki metryk rankingowych na zbiorze testowym w tym istotnie statystycznie (Wilcoxon z korektą Holma-Bonferroniego, $p < 0.001$) poprawiając NDCG@10 o 4,8% względem wariantu logarytmicznego i 6,6% względem liniowego. Osiągnął on również największe pokrycie zbioru wśród modeli implicit ALS.

Badanie pokazało również kompromisy między jakością rankingową, a metrykami wykraczającymi ponad jakość predykcji. Model BPR uzyskał ponad dwukrotnie większe pokrycie katalogu niż najlepszy z modeli iALS oraz najmniejsze obciążenie popularnością, ale osiągnął to kosztem znacznie niższej trafności rekomendacji. Natomiast liniowy model, mimo najniższej jakości rankingowej w rodzinie iALS cechuje się największą różnorodnością rekomendacji oraz zdolnością polecania przedmiotów mniej popularnych niż średnia popularność w zbiorze treningowym danego użytkownika, co jest wysoce nietypowym zachowaniem dla modelu faktoryzacji macierzy.

Przeprowadzona również analiza stabilności ze względu na losowe uruchomienia algorytmów oraz podziały danych ukazały brak znaczących odchyleń modeli. W szczególności odporne okazały się metryki rankingowe, gdzie względne różnice nie przekraczały 0,05%.

Powyższe wyniki pozwalają odpowiedzieć na postawione pytanie badawcze.

7.2. Odpowiedź na pytania badawcze

TODO - na ten moment nie ma gotowego wstępu z zdefiniowanymi pytaniami badawczymi. Niektóre z propozycji to:

- Czy uwzględnienie długości rozgrywki poprawia jakość rekomendacji względem sygnału binarnego?,

- Która funkcja wagi pewności najskuteczniej wykorzystuje informację zawartą w `playtime_forever`?,
- Jak model BPR wypada na tle implicit ALS?,
- Jakie są kompromisy między trafnością a metrykami beyond-accuracy (coverage, diversity, popularity bias)?

7.3. Najważniejsze wnioski praktyczne i metodologiczne

7.3.1. Wnioski praktyczne

Proste przekształcenia wag pewności w przypadku danych o dużym zakresie są podatne na dwa przeciwstawne błędy. Liniowa funkcja jest wrażliwa na wartości odstające, natomiast logarytmiczne przekształcenia są wrażliwe na szum wśród małych wartości. Ten problem wymaga przekształceń z dodatkowym stopniem swobody, takim jak hiperparametr β pozwalającym uwzględnić oba efekty w procesie uczenia.

Sam fakt posiadania produktu w bibliotece stanowi silny sygnał preferencji użytkownika. Rozważanie alternatywnych wag pewności pozwala uzyskać realną, aczkolwiek umiarkowaną poprawę jakości rekomendacji.

Wybór modelu powinien zależeć od pożądanych efektów biznesowych. Jeżeli priorytetem jest jedynie trafność rekomendacji wariant iALS z przesunięciem β jest najlepszym wyborem. W przypadku gdy cenniejsze jest pokrycie katalogu i zmniejszone obciążenie popularnością warto zdecydować się na model BPR. Natomiast gdy wymagany jest kompromis między obiema metrykami rozsądny jest wybór wariantu liniowego.

Redukcja regularyzacji i zwiększenie wymiaru przestrzeni cech ukrytych f może zmniejszać obciążenie popularnością ΔGAP, co zaobserwowano w przypadku wariantu liniowego ALS oraz modelu BPR, osiągających ujemne wartości tej metryki. Nie przekłada się to jednak jednoznacznie na wzrost różnorodności rekomendacji. Model BPR mimo najwyższej liczby czynników ukrytych (96) uzyskał najniższą wartość różnorodności spośród porównywanych modeli, podczas gdy wariant logarytmiczny z przesunięciem β osiągnął wysoką różnorodność przy mniejszej wymiarowości (48). Sugeruje to, że o poziomie obciążenia popularnością decyduje w większym stopniu regularyzacja i charakter funkcji celu. Natomiast różnorodność rekomendacji zależy od tych czynników w sposób niemonotoniczny i wymaga dalszej weryfikacji.

7.3.2. Wnioski metodologiczne

Dokładna analiza wrażliwości hiperparametrów, pozwoliła zinterpretować nie tylko co osiąga dany model, ale również dlaczego otrzymywane są takie wyniki. Dodatkowo wizualizacja funkcji wag pewności umożliwia dodatkowe zrozumienie działania modelu.

Szczególnie istotna okazała się konieczność wielowymiarowej oceny modeli rekomendacyjnych uwzględniająca zarówno metryki trafności jak i metryk wykraczające poza nie. Poleganie wyłącznie na NDCG@10 daje niepełny obraz jakości systemu rekomendacyjnego.

Konieczne są również testy stabilności modelu ze względu na losowość podziału danych oraz inicjalizacji algorytmów jak i testy istotności statystycznej różnic, aby uniknąć błędnych wniosków wynikających z szumu losowego.

7.4. Kierunki dalszych badań

Wyniki metryk ΔGAP i pokrycia pokazały, że modele ALS (poza wariantem liniowym) mają tendencję do faworyzowania popularnych pozycji. Potencjalnym rozwiązaniem byłoby wprowadzenie do funkcji c_{ui} lub do funkcji straty czynnika odwrotnie proporcjonalnego do popularności danego przedmiotu. Metody tego typu są opisywane w literaturze jako debiasing z wykorzystaniem odwrotnego ważenia prawdopodobieństwa ekspozycji (ang. *inverse propensity scoring*, IPS), gdzie waga interakcji jest skalowana odwrotnie do prawdopodobieństwa, że dany przedmiot zostałby zaobserwowany przez użytkownika niezależnie od jego rzeczywistych preferencji [22]. W praktyce prawdopodobieństwo to jest zwykle estymowane na podstawie popularności przedmiotu, ponieważ użytkownicy z większym prawdopodobieństwem wchodzą w interakcję z przedmiotami popularnymi niezależnie od indywidualnych preferencji.

Taka transformacja mogłaby podnosić wagę interakcji z niszowymi grami, kompensując mechanizm, przez który obecne funkcje wag marginalizują rzadkie obserwacje. Warto jednak zaznaczyć, że metody IPS są w praktyce trudne do poprawnego zastosowania – ich nieobciążoność zależy od jakości estymacji prawdopodobieństwa ekspozycji, a niepoprawnie oszacowane wagi mogą prowadzić do wysokiej wariancji i niestabilności procesu uczenia, szczególnie dla przedmiotów o skrajnie niskiej popularności.

Warto zbadać, czy takie podejście pozwoliłoby jednocześnie poprawić pokrycie oraz ΔGAP bez utraty NDCG. Wariant z przesunięciem β pokazał, że jest to możliwe do pewnego stopnia, nadając większą bazową wagę wszystkim interakcjom niezależnie od ich popularności, co jednocześnie poprawiło trafność, pokrycie katalogu oraz obciążenie popularnością względem pozostałych wariantów ALS. Sugeruje to, że połączenie przekształcenia opartego na czasie rozgrywki z jawnym czynnikiem odwrotnie proporcjonalnym do popularności mogłoby przynieść dalszą poprawę tych metryk, choć wymagałoby to starannego doboru estymatora propensity oraz kontroli wariancji wynikającej z ważenia, np. poprzez zastosowanie estymatorów samo-normalizujących (SNIPS).

Poza modyfikacją samej funkcji wag pewności, alternatywnym kierunkiem jest wykorzystanie dopełniających się właściwości testowanych rodzin modeli poprzez ich połączenie w model hybrydowy. Przykładowo model BPR generowałby szeroki wachlarz kandydatów ze względu na większe pokrycie zbioru, a następnie przeprowadzany byłby ponowny ranking przez model iALS cechujący się większą trafnością.

Poza rozszerzeniami wynikającymi z głównych wniosków pracy, warto również rozważyć kierunki badań wskazane jako ograniczenia przyjętej metodologii. Należy do nich segmentacja użytkowników według liczby posiadanych interakcji, umożliwiająca zweryfikowanie hipotezy o wyższej precyzji wariantu liniowego w przypadku użytkowników o większej aktywności. Podobnie, przeprowadzenie eksperymentu z ustaloną, jednakową liczbą czynników ukrytych dla wszystkich wariantów pozwoliłoby rozdzielić wpływ wymiarowości przestrzeni latentnej od wpływu samej funkcji wagowej na wartości metryk różnorodności i pokrycia, co w obecnym układzie eksperymentu pozostaje nierozstrzygnięte ze względu na niezależny dobór tego hiperparametru dla każdego z modeli. Interesującym kierunkiem byłoby także porównanie dobrze dostrojonego wariantu implicit ALS z modelami neuronowymi lub głębokimi, nawiązujące do wyników przedstawionych w pracy [20], w celu sprawdzenia, czy odpowiednio zmodyfikowany model iALS jest w stanie konkurować z bardziej złożonymi

architekturami rekomendacyjnymi. Dodatkowo, zastosowanie pełnej walidacji krzyżowej zamiast pojedynczego podziału na zbiór testowy stanowiłoby naturalne rozszerzenie przeprowadzonej analizy stabilności wyników. Ostatnim istotnym kierunkiem jest walidacja generalizowalności uzyskanych wyników na innych zbiorach danych i domenach niż analizowany zbiór danych , co pozwoliłoby ustalić, czy zaobserwowana przewaga wariantu z przesunięciem β jest właściwością uniwersalną, czy wynika z charakterystyki popularności i rozkładu długości interakcji w badanym zbiorze.

Bibliografia

[1] Himan Abdollahpouri, Masoud Mansoury, Robin Burke i Bamshad Mobasher. „The Unfairness of Popularity Bias in Recommendation". W: *Proceedings of the 13th ACM Conference on Recommender Systems (RecSys '19)*. New York, NY, USA: Association for Computing Machinery, 2019, s. 518–519. DOI: 10.1145/3298689.3347014.

[2] Charu C. Aggarwal. *Recommender Systems: The Textbook*. Cham, Switzerland: Springer, 2016. ISBN: 978-3-319-29657-9. DOI: 10.1007/978-3-319-29659-3.

[3] Takuya Akiba, Shotaro Sano, Toshihiko Yanase, Takeru Ohta i Masanori Koyama. „Optuna: A Next-generation Hyperparameter Optimization Framework". W: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2019, s. 2623–2631. DOI: 10.1145/3292500.3330701.

[4] Apache Spark Project. *Apache Spark Machine Learning Library (MLlib) - Collaborative Filtering*. 2026. URL: https://spark.apache.org/docs/latest/api/python/reference/api/pyspark.ml.recommendation.ALS.html.

[5] Ricardo Bunga, Fernando Batista i Ricardo Ribeiro. „From Implicit Preferences to Ratings: Video Games Recommendation based on Collaborative Filtering". W: *Proceedings of the 13th International Conference on Knowledge Discovery and Information Retrieval*. 2021. DOI: 10.5220/0010655900003064.

[6] Germán Cheuque, Javier Guzmán i Denis Parra. „Recommender Systems for Online Video Game Platforms: The Case of STEAM". W: *Companion Proceedings of The 2019 World Wide Web Conference*. 2019, s. 763–771. DOI: 10.1145/3308560.3316457.

[7] Paolo Cremonesi, Yehuda Koren i Roberto Turrin. „Performance of Recommender Algorithms on Top-N Recommendation Tasks". W: *Proceedings of the Fourth ACM Conference on Recommender Systems*. RecSys '10. New York, NY, USA: ACM, 2010, s. 39–46. ISBN: 978-1-60558-906-0. DOI: 10.1145/1864708.1864721.

[8] Ben Frederickson. *implicit: Fast Python Collaborative Filtering for Implicit Datasets*. GitHub repository. 2022. URL: https://github.com/benfred/implicit.

[9] Xiangnan He, Lizi Liao, Hanwang Zhang, Liqiang Nie, Xia Hu i Tat-Seng Chua. „Neural Collaborative Filtering". W: *Proceedings of the 26th International Conference on World Wide Web*. International World Wide Web Conferences Steering Committee, 2017, s. 173–182. DOI: 10.1145/3038912.3052569.

[10] Yifan Hu, Yehuda Koren i Chris Volinsky. „Collaborative Filtering for Implicit Feedback Datasets". W: *Proceedings of the 8th IEEE International Conference on Data Mining*. 2008, s. 263–272. DOI: 10.1109/ICDM.2008.22.

[11] Dietmar Jannach, Markus Zanker, Alexander Felfernig i Gerhard Friedrich. *Recommender Systems: An Introduction*. Cambridge University Press, 2011. DOI: 10.1017/CBO9780511763113.

[12] Wang-Cheng Kang i Julian McAuley. „Self-Attentive Sequential Recommendation". W: *IEEE International Conference on Data Mining*. 2018, s. 197–206. DOI: 10.1109/ICDM.2018.00035.

[13] Yehuda Koren, Robert Bell i Chris Volinsky. „Matrix Factorization Techniques for Recommender Systems". W: *Computer* 42.8 (2009), s. 30–37. DOI: 10.1109/MC.2009.263.

[14] Yang Li, Kangbo Liu, Ranjan Satapathy, Suhang Wang i Erik Cambria. „Recent Developments in Recommender Systems: A Survey". W: *CoRR* abs/2306.12680 (2023). DOI: 10.48550/arXiv.2306.12680. URL: https://arxiv.org/abs/2306.12680.

[15] Dawen Liang, Rahul G. Krishnan, Matthew D. Hoffman i Tony Jebara. „Variational Autoencoders for Collaborative Filtering". W: *Proceedings of the 2018 World Wide Web Conference (WWW '18)*. ACM, 2018, s. 687–696. DOI: 10.1145/3178876.3186150.

[16] Julian McAuley i UCSD. *Steam Video Game and Bundle Data*. https://cseweb.ucsd.edu/~jmcauley/datasets.html. Dostęp: 2026-06-03.

[17] Eneldo Martínez Mencía. *Evaluation of Recommender Systems in Implicit Feedback Scenarios*. Sprawozdanie techniczne. École Polytechnique Fédérale de Lausanne (EPFL), 2016. URL: https://infoscience.epfl.ch/server/api/core/bitstreams/819ed007-0113-4904-8f01-33488b11c3aa/content.

[18] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner i Lars Schmidt-Thieme. „BPR: Bayesian Personalized Ranking from Implicit Feedback". W: *Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence*. 2009, s. 452–461.

[19] Steffen Rendle, Walid Krichene, Li Zhang i John Anderson. „Neural Collaborative Filtering vs. Matrix Factorization Revisited". W: *arXiv preprint arXiv:2005.09683* (2020). arXiv: 2005.09683.

[20] Steffen Rendle, Walid Krichene, Li Zhang i Yehuda Koren. „Revisiting the Performance of iALS on Item Recommendation Benchmarks". W: *arXiv preprint arXiv:2110.14037* (2021).

[21] Francesco Ricci, Lior Rokach i Bracha Shapira, red. *Recommender Systems Handbook*. 2nd. New York, NY: Springer, 2015. ISBN: 978-1-4899-7636-9. DOI: 10.1007/978-1-4899-7637-6.

[22] Tobias Schnabel, Adith Swaminathan, Ashudeep Singh, Navin Chandak i Thorsten Joachims. „Recommendations as Treatments: Debiasing Learning and Evaluation". W: *Proceedings of the 33rd International Conference on Machine Learning (ICML)*. 2016, s. 1670–1679.

[23] Fei Sun, Jia Liu, Jian Wu, Changhua Pei, Xiao Lin, Wenwu Ou i Peng Jiang. „BERT4Rec: Sequential Recommendation with Bidirectional Encoder Representations from Transformer". W: *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*. 2019, s. 1441–1450. DOI: 10.1145/3357384.3357895.

Wykaz symboli i skrótów

Spis rysunków

4.1 Rozkład dodatniego czasu gry w skali logarytmicznej w przygotowanym zbiorze 30
4.2 Rozkład liczby interakcji przypadających na użytkownika w przygotowanym zbiorze danych . . . 31
4.3 Koncentracja interakcji według popularności gier w przygotowanym zbiorze danych . . . 32
4.4 Dwadzieścia najpopularniejszych gier według liczby interakcji w przygotowanym zbiorze danych . . . 33

6.1 Wrażliwość modeli na hiperparametr α . . . 39
6.2 Wrażliwość modelu na hiperparametr β . . . 40
6.3 Zakres przyjmowanych wartości wag pewności . . . 41
6.4 Wartości wag pewności w zależności od zmiennej `playtime_forever` . . . 42
6.5 Wartości NDCG@10 każdego z modeli na zbiorze testowym . . . 43
6.6 Wartości precyzji względem odtworzenia na zbiorze testowym . . . 44
6.7 Wartości pokrycia katalogu względem NDCG@10 na zbiorze testowym . . . 45
6.8 Średnie wartości różnorodności względem NDCG@10 na zbiorze testowym . . . 46
6.9 Średnie wartości ΔGAP względem NDCG@10 na zbiorze testowym . . . 47

Spis tabel

4.1 Podstawowe statystyki surowego zbioru danych 29
4.2 Wpływ kolejnych etapów przygotowania danych na rozmiar zbioru 29
4.3 Rozkład liczby interakcji użytkowników i popularności gier w przygotowanym zbiorze 31

6.1 Najlepsze hiperparametry wariantów ALS wybrane na zbiorze walidacyjnym 38
6.2 Najlepsze hiperparametry modelu BPR wybrane na zbiorze walidacyjnym . 40
6.3 Wyniki modeli na zbiorze testowym 43
6.4 Wyniki ewaluacji modeli na zbiorze testowym w podziale na metryki dokładności oraz metryki profilowe i różnorodności 48
6.5 Wyniki ewaluacji modeli na zbiorze walidacyjnym w podziale na metryki dokładności oraz metryki profilowe i różnorodności 48